QUANTSIM: A HIGH-FREQUENCY TRADING SIMULATION PLATFORM

A PROJECT REPORT SUBMITTED
IN PARTIAL FULFILMENT OF THE REQUIREMENT
FOR THE AWARD OF THE DEGREE
OF

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & ENGINEERING

BY

ABHISHEK ANAND **22050440004**

AMIK AFFAN **22050440017**

SHAKIR AHMAD **22050440090**

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CAMBRIDGE INSTITUTE OF TECHNOLOGY

TATISILWAI, RANCHI – 835103

JHARKHAND, INDIA

[2026]

DECLARATION CERTIFICATE

This is to certify that the work presented in the project entitled **"QuantSim: A High-Frequency Trading Simulation Platform"** in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering at Cambridge Institute of Technology, Tatisilwai, Ranchi is an authentic work carried out under my supervision and guidance.

To the best of my knowledge, the content of this project does not form a basis for the award of any previous Degree to anyone else.

Date: _____

PROF. DEEPAK KUMAR VERMA
Dept. of Computer Science & Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

PROF. DEEPAK KUMAR VERMA
(Head of Department)
Dept. of Computer Science & Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

CERTIFICATE OF APPROVAL

The foregoing project entitled "QuantSim: A High-Frequency Trading Simulation Platform" is hereby approved as a creditable work on the topic and has been presented satisfactorily to warrant its acceptance as a prerequisite to the degree for which it has been submitted.

It is understood that by this approval, the undersigned does not necessarily endorse any conclusion drawn or opinion expressed therein, but approves the project for the purpose for which it is submitted.

(Internal Examiner) **(External Examiner)**

HEAD OF DEPARTMENT
Dept. of Computer Science and Engineering
Cambridge Institute of Technology
Tatisilwai, Ranchi – 835103

ACKNOWLEDGEMENT

I take this opportunity to thank all who have contributed towards shaping this Final Year Project Report. I would like to express my sincere thanks to Prof. Deepak Kumar Verma (HOD, Department of Computer Science & Engineering) and to my guide Prof. Deepak Kumar Verma, whose help in the course of development of this manuscript is invaluable. As my supervisor, he/she has constantly encouraged me to remain focused on achieving my goal. I would also like to thank the technical staff members of the Department who have been kind enough to advise and help in their respective roles. I have been fortunate to have a wonderful support structure among fellow students. My sincere thanks to everyone who has provided me with kind words, new ideas, useful criticism, or their valuable time. I am truly indebted. Last but not least, I would like to dedicate this work to my family for their love and understanding.

Abhishek Anand
Roll No. 22050440004

ABSTRACT

High-Frequency Trading (HFT) depends on execution engines with sub-microsecond latency and faithful market-microstructure modelling, yet the mechanisms by which leading quantitative firms operate, limit order books, matching engines, queue priority, and inventory-aware market making, remain hidden inside proprietary, costly infrastructure. Compounding this, most accessible backtesting frameworks operate on aggregated Open-High-Low-Close (OHLC) candle data, ignoring the bid-ask spread, price-time priority, queue position, partial fills, transaction costs, and latency. These simplifications systematically overstate strategy performance and produce backtests that fail to generalize to live markets.

This dissertation presents QUANTSIM, an event-driven HFT simulation and backtesting platform that closes these gaps. The system is built around a low-latency C++ core exposing a full Limit Order Book (LOB) with First-In-First-Out (FIFO) price-time priority and a synchronous matching engine supporting eight order types (Limit, Market, IOC, FOK, Post-Only, Stop, Stop-Limit, Iceberg). A realistic execution layer models probabilistic maker fills, queue position, slippage, fees and rebates, borrow costs, and configurable latency, and a pre-trade risk engine adds a lock-free kill switch, rate limiter, position and notional limits, and a stale-price guard.

The platform ships five strategies, Avellaneda-Stoikov market making, Ornstein-Uhlenbeck pairs trading, TWAP execution, a multi-asset mean-reversion portfolio, and a Black-Scholes options market-maker with delta hedging, and bridges its C++ core to Python via pybind11 so that strategies and analytics are authored in Python while executing on the low-latency engine. A GPU-rendered Dear ImGui/ImPlot interface provides thirteen analytical tabs, an in-application C++ strategy compiler, and a Bookmap-style liquidity heatmap, and the same strategy binary runs unchanged in both the backtester and a C++-native live Binance paper-trading harness.

Empirical evaluation shows a matching-engine throughput above 14 million operations per second at a median latency below 100 nanoseconds, with all 27 unit tests passing. QUANTSIM thus unifies order-book-level realism, reproducibility, and interactive analysis in one free, open, and extensible platform, giving students and researchers direct access to execution mechanics previously confined to proprietary systems.

Keywords: High-Frequency Trading, Limit Order Book, Matching Engine, Market Microstructure, Avellaneda-Stoikov, Statistical Arbitrage, Event-Driven Backtesting, Low-Latency C++, pybind11, Dear ImGui.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	**Full Form**
ABI	Application Binary Interface
API	Application Programming Interface
AS	Avellaneda-Stoikov (market-making model)
ATM	At-The-Money
BS	Black-Scholes
CARA	Constant Absolute Risk Aversion
CLI	Command-Line Interface
CPU	Central Processing Unit
CSV	Comma-Separated Values
DFD	Data Flow Diagram
DMA	Direct Market Access
FIFO	First-In, First-Out
FOK	Fill-Or-Kill
FPS	Frames Per Second
GBM	Geometric Brownian Motion
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HFT	High-Frequency Trading
IOC	Immediate-Or-Cancel
IS	Implementation Shortfall

Abbreviation	Full Form
ITCH	Nasdaq market-data binary protocol
IV	Implied Volatility
JSON	JavaScript Object Notation
LOB	Limit Order Book
MM	Market Making
OHLC	Open-High-Low-Close
OU	Ornstein-Uhlenbeck (process)
PnL	Profit and Loss
RNG	Random Number Generator
RSI	Relative Strength Index
SIMD	Single Instruction, Multiple Data
SMA	Simple Moving Average
TCA	Transaction Cost Analysis
TLS	Transport Layer Security
TWAP	Time-Weighted Average Price
VaR	Value at Risk
VWAP	Volume-Weighted Average Price
WSS	WebSocket Secure

Chapter 1

INTRODUCTION

1.1 Background and Motivation

Financial markets have undergone a profound structural transformation over the past three decades. The migration from open-outcry trading floors to fully electronic, order-driven exchanges has compressed the timescale of trading from seconds to microseconds and, in the most competitive venues, to nanoseconds. Within this electronic ecosystem, High-Frequency Trading (HFT) has emerged as a dominant force, accounting for a substantial fraction of the daily volume on major equity, futures, and cryptocurrency exchanges. HFT firms deploy automated strategies that submit, modify, and cancel millions of orders per day, each holding positions for fractions of a second and seeking to capture vanishingly small but statistically reliable profits across enormous trade counts.

The defining characteristics of HFT are ultra-low-latency execution, an extremely high order-to-fill ratio, the systematic avoidance of overnight position risk, and the extraction of small per-trade margins amplified by volume. Success in this domain depends critically on a precise understanding of *market microstructure*, the body of theory and practice concerned with how the specific rules of an exchange, the available order types, and the behaviour of competing participants shape the process of price discovery. At the heart of every modern electronic exchange lies the Limit Order Book (LOB): a continuously updated, two-sided ledger of all outstanding buy and sell limit orders, organized by price and, within each price, by time of arrival.

Despite the central importance of microstructure, the tools available to students, researchers, and aspiring quantitative traders for studying these phenomena are markedly inadequate. The overwhelming majority of accessible backtesting frameworks operate on aggregated Open-High-Low-Close (OHLC) candle data sampled at intervals of one minute or longer. Such frameworks implicitly assume that an order is filled instanta-

neously at the candle's closing price, with no spread to cross, no queue to wait in, no partial fills, no market impact, and no latency between signal and execution. These assumptions are not merely simplifications; they are structurally incompatible with the economics of high-frequency strategies. A market-making strategy, for example, earns its profit precisely from the bid-ask spread and from its position in the order queue, the very quantities that candle-based backtesting erases. As a result, candle-based backtests routinely report Sharpe ratios that are unattainable in practice, producing a dangerous gap between simulated and realized performance.

A second barrier is one of access and cost. Historically, the infrastructure required for serious HFT research, co-located servers, direct market access, and tick-by-tick data feeds such as Nasdaq ITCH or proprietary vendor feeds from Bloomberg and Refinitiv, has been the exclusive preserve of well-capitalized institutional firms. The annual cost of a single professional data feed can exceed the entire budget of a university research group. This economic barrier has effectively excluded the academic community from hands-on study of the mechanisms that now govern modern markets.

The rise of cryptocurrency exchanges has dramatically altered this landscape. Venues such as Binance expose free, public WebSocket endpoints that stream full-depth order book updates and the complete trade tape, tick by tick, with no requirement for an account or an API key for read-only market data. This development has created, for the first time, an open sandbox in which the dynamics of a live, liquid, electronic order book can be observed and experimented upon without prohibitive cost. QUANTSIM was conceived to seize this opportunity: to build a single, coherent platform that combines the microstructural fidelity of an institutional matching engine, the analytical convenience of a Python research environment, and the immediacy of a real-time graphical interface, all validated against free live market data.

The immediate impetus for this project was a concrete and personal one. In seeking to understand how elite quantitative firms such as Jane Street and Citadel actually operate at the level of the order book, the authors drew on practitioner explanations popularized in online lectures [1] and reference glossaries [2], which conveyed the concepts clearly but offered no executable environment in which to manipulate them. No free simulator existed through which a student could see and experiment with these mechanics firsthand. Every adequate tool was either a closed institutional system inaccessible outside a trading firm, or an accessible candle-based backtester that hid precisely the mechanics worth studying. QUANTSIM was built to address this void: a free and open platform, engineered to the standards of a production trading system, that allows students to observe and manipulate the mechanics of quantitative trading directly rather than studying them only in the abstract.

The motivation for this project is therefore fourfold. First, there is a pedagogical motivation: to provide a faithful, transparent environment in which the mechanics of order matching, queue priority, and execution cost can be studied directly rather than abstracted away. Second, there is a research motivation: to enable rigorous, reproducible evaluation of trading strategies under realistic execution assumptions, including walk-forward and purged cross-validation that guard against the overfitting endemic to naive backtesting. Third, there is an engineering motivation: to demonstrate that the performance demanded by high-frequency simulation, millions of order operations per second at sub-microsecond latency, can be achieved within an extensible, well-tested, and approachable codebase rather than an opaque black box. Fourth, and binding the others together, there is a motivation of access: to deliver these capabilities at zero cost and in open form, so that the rigorous study of market microstructure is no longer the exclusive preserve of well-funded institutions but is available to any student or researcher with a laptop.

1.2 Problem Statement

Existing backtesting and trading-simulation tools fail to model the essential mechanics of modern electronic markets. They abstract away the limit order book, ignore price-time priority and queue position, assume instantaneous and costless fills, and disregard the latency between signal generation and order arrival. Consequently, they cannot faithfully evaluate latency-sensitive and microstructure-dependent strategies such as market making and statistical arbitrage, and they systematically overstate achievable performance. There is no widely accessible, open, and extensible platform that simultaneously delivers order-book-level matching-engine realism, a productive research interface for strategy development, real-time interactive visualization, and validation against live market data. This project addresses the absence of such a unified platform.

1.3 Objectives of the Project

The principal objectives of the QUANTSIM project are as follows:

- To design and implement a high-performance Limit Order Book and synchronous Matching Engine in modern C++ that enforces strict price-time (FIFO) priority and supports the full range of institutional order types, including Limit, Market, IOC, FOK, Post-Only, Stop, Stop-Limit, and Iceberg orders.
- To construct an event-driven backtesting framework that models realistic execution effects, probabilistic maker fills, queue position, slippage, exchange fees and

rebates, short-borrow costs, and configurable network latency, in place of the idealized fills assumed by candle-based tools.

- To implement a library of canonical quantitative trading strategies with rigorous mathematical foundations, including the Avellaneda-Stoikov market-making model, Ornstein-Uhlenbeck pairs trading, TWAP execution, a multi-asset mean-reversion portfolio, and a Black-Scholes options market-maker with delta hedging.
- To bridge the C++ core to Python through pybind11, enabling strategy authoring and analytics in a high-level language while retaining low-latency execution, and to provide a comprehensive research toolkit (signal library, walk-forward analysis, purged k-fold cross-validation, and tear-sheet reporting).
- To develop a real-time, GPU-rendered graphical interface that visualizes order-book depth, equity and drawdown curves, a Bookmap-style liquidity heatmap, correlation matrices, and Greeks surfaces, and that embeds an on-the-fly C++ strategy compiler.
- To integrate a pre-trade risk-management subsystem (kill switch, rate limiter, position and notional limits, stale-price guard) and to validate the entire platform against live Binance market data through a C++-native paper-trading harness.

1.4 Scope of the Project

The scope of QUANTSIM encompasses the complete simulation pipeline from market-data ingestion through strategy execution to performance analysis and visualization. On the data side, the platform ingests synthetic feeds (Geometric Brownian Motion, Ornstein-Uhlenbeck spreads, and a multi-asset factor model), historical CSV tick replays, the Nasdaq ITCH 5.0 binary protocol, and live Binance WebSocket streams. On the execution side, it provides a full LOB, a matching engine, and a realistic cost-and-latency model. On the strategy side, it ships a library of built-in strategies and supports user-authored strategies in both Python and C++.

The platform is intended for simulation, research, and paper trading; it is not a production order-routing gateway and does not place capital at risk on live venues. The live integration operates exclusively in paper-trading mode, generating synthetic fills against the real trade tape and a queue model. The matching engine is single-threaded and synchronous by design, prioritizing determinism and reproducibility over the multi-threaded concurrency of a production exchange. The cost and latency models are parameterized approximations rather than exact reproductions of any specific venue's fee schedule or network topology. These boundaries are deliberate: they keep the platform

tractable, deterministic, and pedagogically transparent while preserving the microstructural realism that distinguishes it from candle-based alternatives.

1.5 Organization of the Report

The remainder of this dissertation is organized into five further chapters. Chapter 2 surveys the relevant literature spanning market microstructure theory, optimal market-making models, statistical arbitrage, execution algorithms, and software-engineering practice for low-latency systems; it then presents a comparative analysis of existing tools and identifies the research gap that QUANTSIM addresses. Chapter 3 details the system analysis and design, including functional and non-functional requirements, the layered architecture, data flow diagrams, and the design patterns employed. Chapter 4 describes the implementation in depth, covering the technologies used, the structure of each major module, the core algorithms with pseudo-code, and annotated code samples. Chapter 5 presents the testing strategy, the unit-test suite, performance benchmarks, and empirical strategy results. Chapter 6 concludes the dissertation and outlines directions for future work. The report ends with a list of references in IEEE format and two appendices containing source-code highlights and sample outputs.

Chapter 2

LITERATURE REVIEW

This chapter situates QUANTSIM within the existing body of research and practice. Section 2.1 surveys the foundational and contemporary literature across five themes: market microstructure theory, optimal market-making models, statistical arbitrage, execution algorithms, and the software engineering of low-latency trading systems. Section 2.2 presents a structured comparison of existing simulation and backtesting tools. Section 2.3 synthesizes these findings to articulate the specific research gap that this project addresses.

2.1 Related Works

2.1.1 Market Microstructure Theory

The theoretical study of how trading mechanisms determine prices is surveyed comprehensively by O'Hara [3], who established market microstructure as a distinct discipline synthesizing inventory-based and information-based models of the bid-ask spread. The information-based view was crystallized by Kyle [4], whose model of a strategic informed trader interacting with noise traders and a competitive market maker yields the celebrated linear price-impact relationship and the concept of market depth, both directly relevant to the limit-order-book dynamics that this project simulates.

2.1.2 Optimal Market Making

The canonical model for optimal market making is due to Avellaneda and Stoikov [5], who cast the dealer's problem as one of stochastic optimal control. Assuming a mid-price following arithmetic Brownian motion and an exponential (CARA) utility over terminal wealth, they derive closed-form approximations for the reservation price, the

inventory-adjusted price at which the dealer is indifferent to trading, and for the optimal bid-ask spread as a function of risk aversion, volatility, time horizon, and order-arrival intensity. Their key insight is that the dealer should skew quotes around the reservation price rather than the mid-price, automatically biasing the book to mean-revert inventory toward zero. This model is the foundation of the market-making strategy implemented in QUANTSIM and is developed in detail in Chapter 4.

The Avellaneda-Stoikov framework sits within the broader treatment of Cartea, Jaimungal, and Penalva [6], whose text presents algorithmic and high-frequency trading, market making, optimal execution, and statistical arbitrage within a common stochastic-control language, and is a primary reference for the strategy formulations adopted in this project.

2.1.3 Statistical Arbitrage and Pairs Trading

Statistical arbitrage exploits temporary mispricings between statistically related instruments. The classic pairs-trading approach, documented by Gatev, Goetzmann, and Rouwenhorst [7] in their study of the U.S. equity market, identifies pairs of securities whose prices have historically moved together and trades the spread when it deviates from its historical norm, betting on reversion. The statistical foundation for trading such a spread is the mean-reverting Ornstein-Uhlenbeck (OU) process, which reverts to a long-run mean at a rate set by a mean-reversion speed parameter, together with the related concept of cointegration, which determines when a linear combination of non-stationary price series is itself stationary and therefore tradeable. QUANTSIM implements both a synthetic OU-spread feed for controlled experiments and a z-score-based pairs-trading strategy, described in Chapter 4.

2.1.4 Optimal Execution

When a trader must execute a large order, naive immediate execution incurs substantial market impact. The seminal framework of Almgren and Chriss [8] formalizes the trade-off between market impact (which favours slow execution) and price risk (which favours fast execution), deriving an optimal execution trajectory that balances the two according to the trader's risk aversion. The Time-Weighted Average Price (TWAP) and Volume-Weighted Average Price (VWAP) algorithms are widely used industrial benchmarks for execution quality; the associated performance metric, Implementation Shortfall, captures the total cost of executing a decision relative to the price prevailing when the decision was made. QUANTSIM implements a TWAP execution strategy and reports Implementation Shortfall as described in Chapter 4.

2.1.5 Options Pricing and Greeks

The pricing of options market-making activity in QUANTSIM rests on the Black-Scholes model [9], which provides a closed-form price for a European option under the assumption of geometric Brownian motion for the underlying and continuous, costless hedging. The partial derivatives of the option price with respect to its inputs, collectively the "Greeks" (delta, gamma, vega, theta, rho), quantify the option's sensitivities and are the basis for dynamic hedging. Hull [10] provides the standard reference treatment of options, futures, and the practice of delta hedging that underlies the options strategy implemented here.

2.1.6 Low-Latency Systems Engineering

The performance characteristics of QUANTSIM draw on a body of systems-engineering knowledge concerned with low-latency, cache-conscious software. The general principles of mechanical sympathy, cache-line alignment to avoid false sharing, and lock-free data structures are central to the metrics and risk subsystems of this platform. The C++ language features exploited, `std::map` for ordered price levels, `std::list` for intrusive FIFO queues, `std::atomic` for lock-free counters, and the `std::mdspan` multidimensional view (standardized in C++23, compiled here under the C++26 standard) for the Greeks surface, are documented in the ISO C++ standard and on the cppreference site [11]. The interoperability between C++ and Python is achieved through the pybind11 library [12], which provides a header-only, zero-overhead binding layer. The graphical interface is built on Dear ImGui [13], an immediate-mode GUI library whose stateless, redraw-every-frame architecture is well suited to high-throughput data visualization.

2.2 Comparative Analysis

Several open-source and commercial tools exist for trading-strategy simulation, each occupying a different point in the trade-off space between realism, performance, accessibility, and feature breadth. Table 2.1 compares the most prominent of these against QUANTSIM across the dimensions most relevant to high-frequency research.

Table 2.1. Comparison of trading simulation and backtesting frameworks

Feature	Backtrader	Zipline	Backtesting.py	QUANTSIM
Data granularity	Bar/candle	Daily/minute	Bar/candle	Tick / L2
Limit order book	No	No	No	Yes (FIFO)
Queue priority	No	No	No	Yes
Order types	Basic	Basic	Basic	8 types
Latency model	No	No	No	Yes
Fees/rebates	Partial	Partial	Partial	Maker/taker
Native speed	Python	Python	Python	C++ core
Live integration	Broker API	No	No	WebSocket
Real-time GUI	No	No	Plot only	GPU/ImGui
Options/Greeks	No	No	No	Yes
On-the-fly compile	No	No	No	Yes (C++)

The comparison reveals a consistent pattern. The Python-based frameworks, Backtrader, Zipline, and Backtesting.py, are accessible and feature-rich at the level of portfolio and signal logic, but they operate exclusively on aggregated bar data and model neither the order book, queue priority, nor latency, making them unsuitable for strategies whose economics depend on microstructure. Commercial simulators that do model the order book are typically proprietary, expensive, and closed. QUANTSIM occupies the previously vacant intersection: order-book-level realism with a native C++ engine, while remaining open, extensible, and approachable through a Python interface and an interactive GUI.

2.3 Gap Identification

The survey above exposes a clear gap. On one side stand accessible, open-source backtesting libraries that sacrifice microstructural realism for ease of use; their candle-based models cannot represent the spread capture, queue position, and latency sensitivity that define high-frequency strategies. On the other side stand institutional simulators that capture this realism but are closed, costly, and pedagogically opaque. No widely available platform unifies an order-book-level matching engine with strict price-time priority and a full set of order types, a realistic execution-cost and latency model, a productive research interface, real-time visualization, and validation against live market data, all in a single, open, and extensible system. This gap is precisely what QUANTSIM is designed to fill, combining a native C++ core, a pybind11 bridge, an ImGui front-end, and a WebSocket harness; the chapters that follow describe how this synthesis was designed, implemented, and validated.

Chapter 3

SYSTEM ANALYSIS AND DESIGN

This chapter presents the analysis and design of QUANTSIM. Section 3.1 enumerates the functional and non-functional requirements together with the hardware and software environment. Section 3.2 describes the layered system architecture. Section 3.3 presents the data flow diagrams at two levels of abstraction. Section 3.5 discusses the design patterns and key implementation decisions that give the system its structure.

3.1 System Requirements

3.1.1 Functional Requirements

The functional requirements specify the observable behaviour the system must exhibit. They are summarized in Table 3.1.

Table 3.1. Functional requirements of QUANTSIM

ID	Requirement
FR-1	The system shall maintain a two-sided limit order book with price-time (FIFO) priority for each traded symbol.
FR-2	The matching engine shall support Limit, Market, IOC, FOK, Post-Only, Stop, Stop-Limit, and Iceberg order types.
FR-3	The system shall generate synthetic market data via GBM, OU-spread, and multi-asset factor models, and shall replay CSV and ITCH 5.0 data.
FR-4	The system shall execute built-in strategies (market making, statistical arbitrage, TWAP, portfolio, options) and user-authored strategies.
FR-5	The backtester shall model probabilistic maker fills, queue position, slippage, fees, rebates, borrow cost, and configurable latency.
FR-6	The system shall compute performance metrics including Sharpe, Sortino, Calmar, maximum drawdown, volatility, profit factor, and win rate.
FR-7	The risk engine shall enforce a kill switch, rate limiting, position and notional limits, and a stale-price guard before order submission.
FR-8	The GUI shall render order-book depth, equity and drawdown curves, a liquidity heatmap, correlation matrices, and Greeks surfaces in real time.
FR-9	The system shall compile user-written C++ strategies to shared libraries at runtime and load them without restarting.
FR-10	The system shall connect to the Binance WebSocket feed and execute strategies in paper-trading mode against live data.
FR-11	The system shall export tear-sheet reports in JSON, CSV, and self-contained HTML formats.

3.1.2 Non-Functional Requirements

The non-functional requirements constrain the quality attributes of the system. Chief among them is performance: the matching engine must sustain a throughput on the order of millions of operations per second with sub-microsecond per-order latency, since a high-frequency simulation processing millions of ticks would otherwise be intractable. Reproducibility is equally critical: given the same seed and parameters, a simulation must produce bit-identical results, which mandates a single-threaded, deterministic matching core and a controlled random-number generator. The system must remain responsive, sustaining a 60-frames-per-second interface even while a simulation runs on a background thread; this dictates a clean separation between the simulation thread and the render thread, mediated by a mutex-protected shared state. Finally, the platform must be extensible, new strategies, feeds, and metrics must be addable without modifying the core, and portable across the development platform.

3.1.3 Hardware and Software Environment

Tables 3.2 and 3.3 list the hardware and software environment used for development and evaluation.

Table 3.2. Hardware environment

Component	Specification
Processor	Multi-core x86-64 / ARM64 with AVX support
Memory	8 GB RAM minimum (16 GB recommended)
Graphics	OpenGL 3.3-capable GPU
Storage	2 GB free disk space for build artifacts and data
Network	Broadband connection for live WebSocket feed

Table 3.3. Software environment

Component	Specification
Operating system	macOS (Darwin) / Linux
C++ compiler	Clang or GCC (C++23, with selective C++26 features)
Build system	CMake 3.20 or later
Python	Python 3.10 or later
Python packages	pybind11, NumPy, pandas, matplotlib, yfinance
GUI libraries	Dear ImGui, ImPlot, GLFW, OpenGL
Networking	OpenSSL (for live WebSocket-over-TLS)

3.2 System Architecture

QUANTSIM adopts a layered architecture that cleanly separates the low-latency execution core from the research, presentation, and live-trading layers. Figure 3.1 depicts the four principal layers and the flow of control and data between them. At the foundation sits the C++ core, comprising the limit order book, the matching engine, the market-data feeds, the risk engine, and the metrics subsystem. Above this, the pybind11 bridge exposes the core to Python, where the research layer provides strategies, signals, analytics, and validation. The presentation layer, the ImGui desktop application, communicates with the core through a mutex-protected shared state populated by a background simulation runner. A parallel live-trading path connects the same strategy binaries, via a C++-native WebSocket client, to the Binance feed.

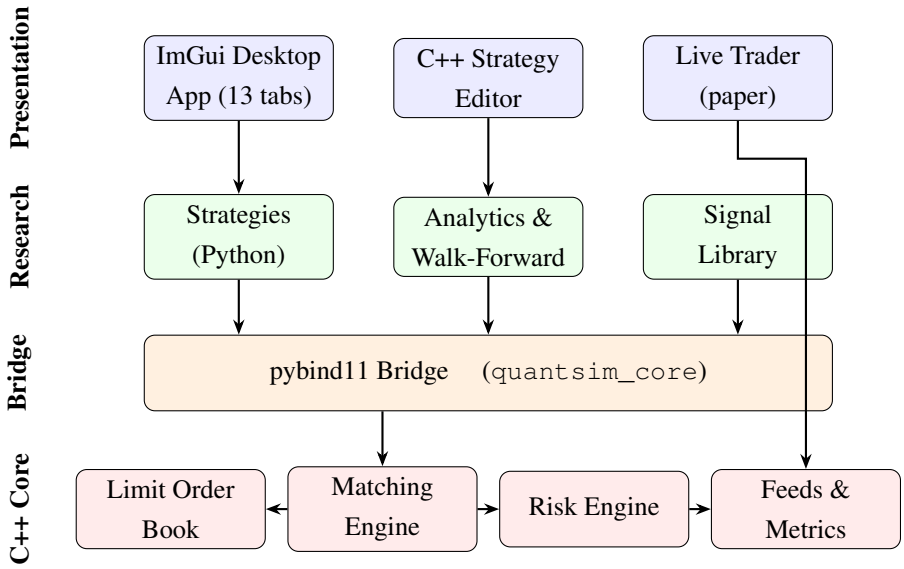

Figure 3.1. Four-layer architecture of QUANTSIM. The C++ core provides low-latency execution; the pybind11 bridge exposes it to the Python research layer; the ImGui presentation layer visualizes results; the live trader reuses core feeds and strategy binaries.

The rationale for this layering is the separation of concerns by performance requirement. Components on the critical execution path, order matching, risk checks, metrics, reside in the C++ core where every nanosecond matters. Components on the productivity path, strategy logic, statistical analysis, plotting, reside in Python where developer velocity matters more than raw speed. The pybind11 bridge mediates the two, paying a marshalling cost only at the boundary, which is amortized by batch interfaces for the hottest operations. The presentation layer never touches the core directly; it reads a snapshot of shared state under a mutex, decoupling render cadence from simulation cadence entirely.

3.3 Data Flow Diagrams

3.3.1 Level 0 (Context Diagram)

The Level 0 data flow diagram, shown in Figure 3.2, treats the entire QUANTSIM platform as a single process and identifies its external entities and the data that flows between them. The user supplies strategy definitions and configuration parameters and receives performance metrics, charts, and reports. Market-data sources, synthetic generators, historical files, and the live Binance feed, supply tick and order-book data. The file system receives exported reports and supplies historical data and compiled strategy binaries.

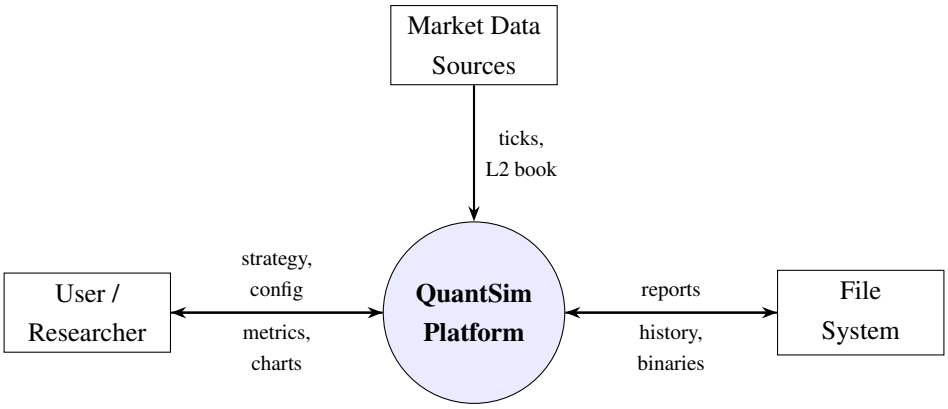

Figure 3.2. Level 0 (context) data flow diagram. The platform exchanges configuration and results with the user, ingests market data, and reads from and writes to the file system.

3.3.2 Level 1 (Detailed Diagram)

The Level 1 diagram, Figure 3.3, decomposes the platform into its principal processes and data stores. A tick from a feed enters the simulation runner, which forwards the order-book state to the active strategy. The strategy emits orders, which pass through the risk engine before reaching the matching engine. The matching engine updates the order book and emits fills, which the metrics process consumes to update positions, PnL, and the equity curve. The shared state store mediates between the simulation thread and the rendering process.

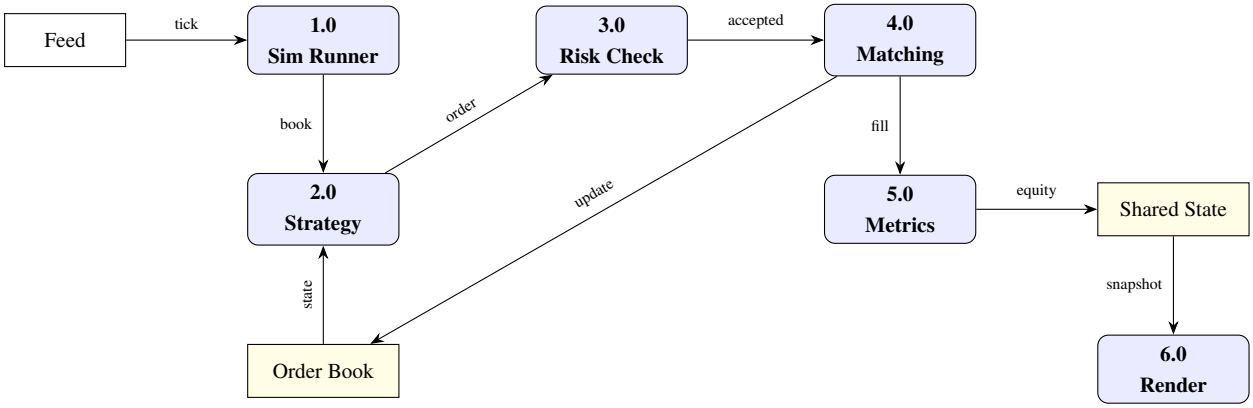

Figure 3.3. Level 1 data flow diagram. Ticks drive the simulation runner; orders flow through the risk engine to the matching engine; fills update metrics and the shared state, which the render process reads.

3.4 UML and ER Diagrams

Because QUANTSIM is a computational platform rather than a database-centric application, its structure is best captured by behavioural and structural UML diagrams rather

than an entity-relationship schema. This section presents a use-case diagram describing the interactions available to the user, a class diagram of the core engine, and a sequence diagram tracing the lifecycle of an order. A simplified entity-relationship view of the platform's persistent data is also given for completeness.

3.4.1 Use Case Diagram

Figure 3.4 shows the principal use cases of the platform from the perspective of two actors: the researcher, who configures and runs simulations and analyses results, and the external market-data feed, which supplies ticks to the running simulation. The researcher can select or author a strategy, configure run parameters, execute a backtest, inspect results and metrics, compile a custom C++ strategy, and launch a live paper-trading session.

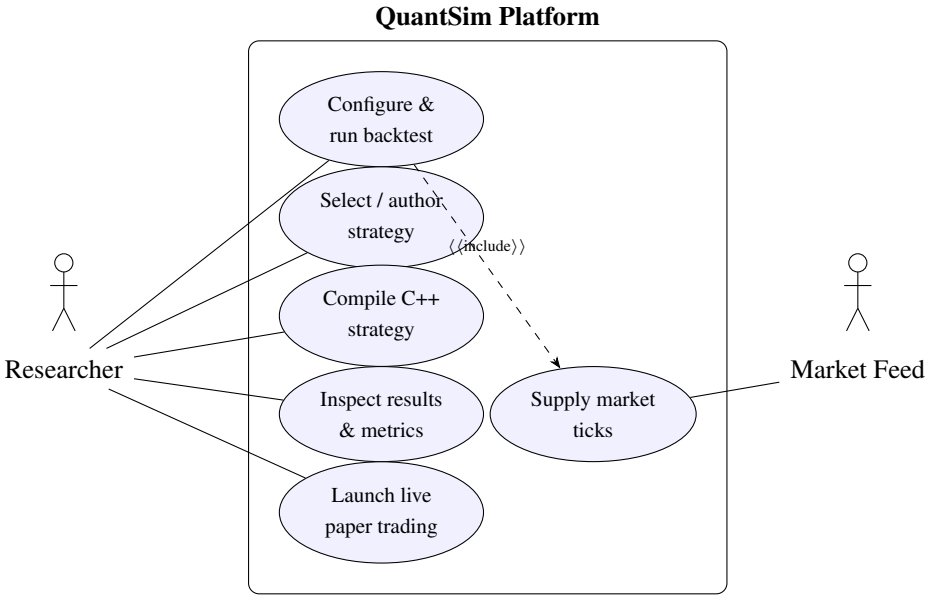

Figure 3.4. Use case diagram. The researcher drives configuration, execution, and analysis; the market feed supplies ticks consumed during a run.

3.4.2 Class Diagram

Figure 3.5 presents a class diagram of the C++ core. The `MatchingEngine` aggregates one `LimitOrderBook` per symbol and consults the `PreTradeRisk` engine before accepting an order; each book is composed of `PriceLevel` objects that in turn hold `Order` objects in FIFO queues. Matching produces `Fill` objects that are recorded by `TradingMetrics`. The `SimRunner` orchestrates the run, pulling ticks from a `Feed` and dispatching them to the active `Strategy`.

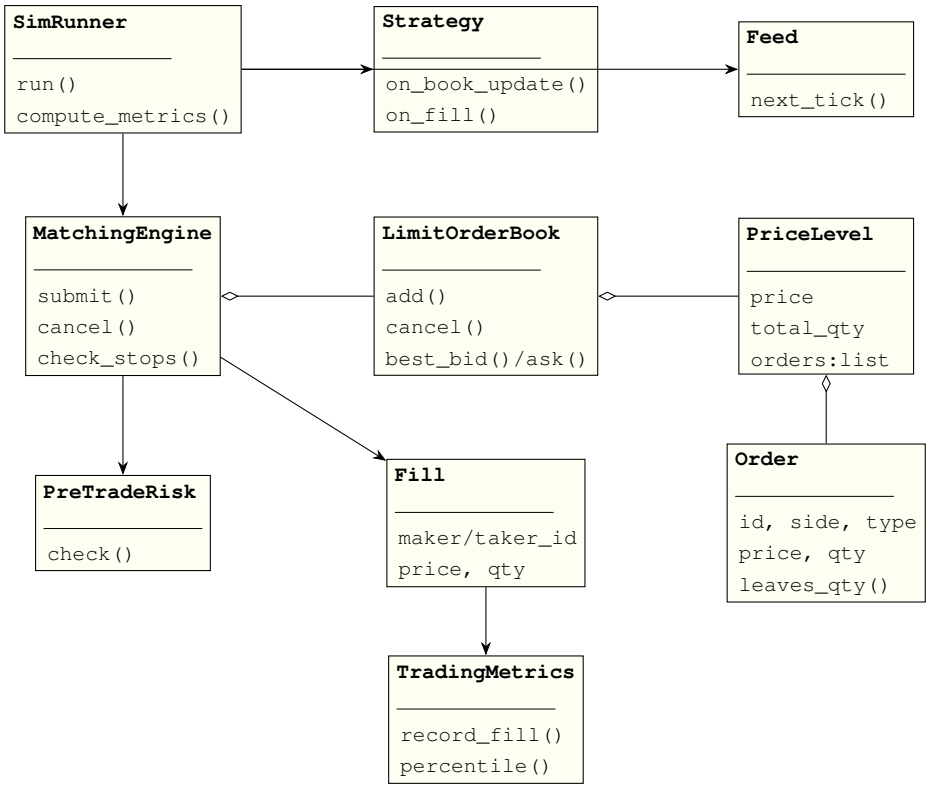

Figure 3.5. Class diagram of the C++ core. Diamonds denote composition/aggregation; arrows denote dependency.

3.4.3 Sequence Diagram

Figure 3.6 traces the lifecycle of a single order through the system. A tick arrives from the feed and is handed by the simulation runner to the strategy, which decides to submit an order. The order passes through the risk engine, then to the matching engine, which matches it against the book and emits a fill; the fill propagates back to the metrics collector and ultimately to the shared state read by the interface.

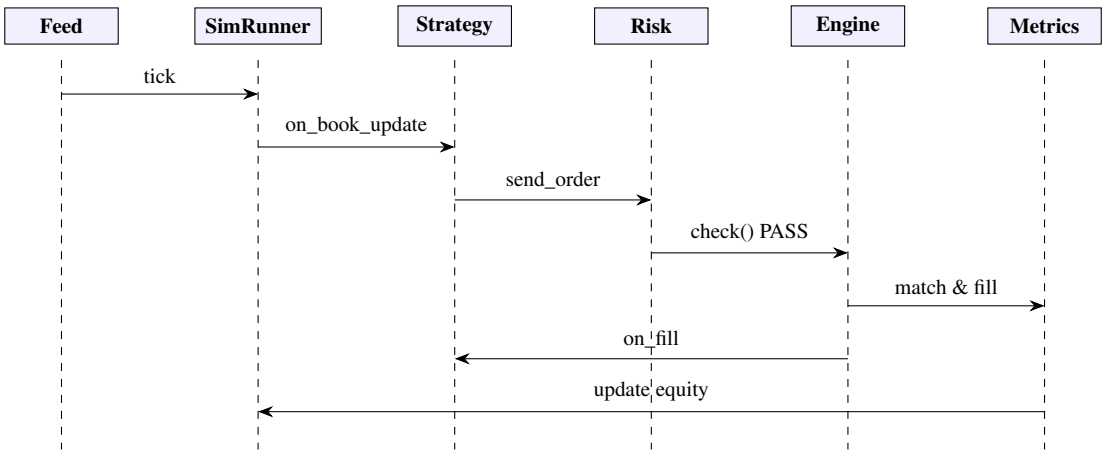

Figure 3.6. Sequence diagram for the order lifecycle, from tick arrival through risk check and matching to fill notification and equity update.

3.4.4 Entity-Relationship View

Although QUANTSIM is computation-centric and holds most state in memory, its persisted artifacts, exported tear sheets and recorded runs, can be described by the simple entity-relationship view of Figure 3.7. A RUN is parameterized by a STRATEGY and a FEED configuration and produces many FILL records and a single METRICSREPORT.

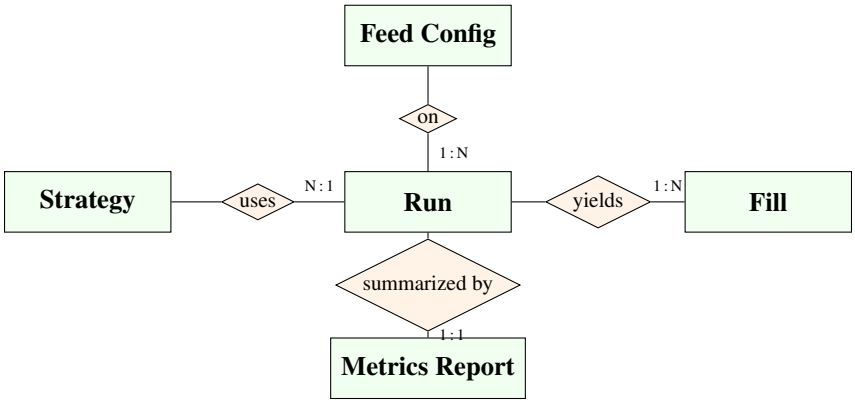

Figure 3.7. Entity-relationship view of the platform's persisted artifacts. A run uses a strategy and feed configuration, yields many fills, and is summarized by one metrics report.

3.5 Design Patterns and Implementation Decisions

The design of QUANTSIM embodies several recognized design patterns and deliberate structural decisions. The matching engine employs the **Observer pattern** through a set of callbacks, `on_fill`, `on_add`, `on_cancel`, and `on_book_update`, that decouple the engine from the consumers of its events. A strategy, a metrics collector, and a logger can all subscribe to fills without the engine knowing anything about them, which keeps the core free of presentation and accounting concerns.

The strategy subsystem follows the **Strategy pattern** (in both the design-pattern and the finance sense). Each trading strategy implements a common interface, in Python an abstract base class with an `on_book_update` method, and in C++ a stable Application Binary Interface (ABI) of `extern "C"` entry points. Because the interface is fixed, the engine can drive any strategy polymorphically, and the same compiled C++ strategy binary runs unchanged in both the backtester and the live trader. This is the single most important architectural decision in the system: it guarantees that what is tested in simulation is exactly what executes against live data, eliminating the backtest-to-live divergence that plagues many trading systems.

The order book uses an **intrusive container design**: each price level holds a `std::list` of order pointers providing the FIFO queue, while a separate hash map indexes every live order by its identifier, storing an iterator into the appropriate list. This dual structure

yields $O(\log N)$ price lookup through the ordered map of price levels and $O(1)$ cancellation through the hash-map-to-iterator indirection, combining the ordering guarantees of a balanced tree with the constant-time removal of a linked list.

The shared state between the simulation and render threads is protected by a single mutex and accessed through a **snapshot discipline**: the render thread acquires the lock only briefly to copy the data it needs, then releases it before drawing. This keeps lock contention negligible and ensures the user interface never blocks the simulation. Finally, the metrics and risk subsystems use **cache-line alignment** (`alignas(64)`) and lock-free atomics to keep the hot path free of false sharing and lock overhead, a decision justified by the performance benchmarks reported in Chapter 5.

Chapter 4

IMPLEMENTATION

This chapter describes the implementation of QUANTSIM in detail. Section 4.1 justifies the choice of technologies. Section 4.2 describes each major module. Section 4.3 presents the core algorithms with pseudo-code and mathematical formulations. Section 4.4 provides annotated code samples of the most important components.

4.1 Technologies and Tools Used

The implementation language for the core is modern C++ (C++23, with selective use of C++26 features). C++ was chosen because high-frequency simulation is fundamentally a performance problem: processing millions of order events per second at sub-microsecond latency is achievable in a systems language with manual memory control, zero-cost abstractions, and predictable performance, but not in a managed or interpreted runtime. The core is largely header-only, which enables aggressive inlining and link-time optimization and simplifies the build.

Python serves as the research and strategy-authoring language. Its rich scientific ecosystem, NumPy for vectorized numerics, pandas for tabular analysis, and matplotlib for plotting, makes it the natural environment for statistical work, and its low ceremony accelerates strategy iteration. The two languages are joined by **pybind11**, a header-only binding library that exposes C++ classes and functions to Python with negligible overhead and automatic type conversion.

The graphical interface is built on **Dear ImGui** [13] and its plotting companion **ImPlot**, rendered through **GLFW** [14] and **OpenGL**. ImGui's immediate-mode paradigm, in which the entire interface is redrawn from scratch every frame directly on the GPU, avoids the retained-mode bookkeeping that makes traditional toolkits struggle under high-frequency data streams. The live-trading harness uses **OpenSSL** [15] to imple-

ment a WebSocket-over-TLS client with no heavyweight dependencies. The build is orchestrated by **CMake** [16], and the C++ code is compiled with `-O3 -march=native` to enable full optimization and auto-vectorization. Table 4.1 summarizes the technology stack and the justification for each choice.

Table 4.1. Technology stack and justifications

Technology	Role	Justification
C++23/26	Core engine	Sub-microsecond latency, zero-cost abstractions
Python 3.10+	Research, strategies	Scientific ecosystem, rapid iteration
pybind11	Language bridge	Header-only, low-overhead bindings
Dear ImGui / ImPlot	GUI	Immediate-mode, GPU-rendered, high throughput
GLFW / OpenGL	Windowing / render	Cross-platform, lightweight
OpenSSL	Live WSS client	TLS without heavyweight dependencies
CMake	Build system	Cross-platform, dependency management
NumPy / pandas	Analytics	Vectorized numerics, tabular analysis

4.2 Module Description

4.2.1 Order and Fill Representation

The fundamental data types are defined in `order.hpp`. The `Order` structure carries a unique identifier, the trading symbol, the side (buy or sell), the order type, the limit price, the requested and filled quantities, the status, a nanosecond timestamp, and auxiliary fields for advanced order types (a stop trigger price and a display quantity for icebergs). Two derived quantities, the leaves quantity (unfilled remainder) and a done predicate, are computed inline. The `Fill` structure records a matched trade: the maker and taker identifiers, the execution price and quantity, the aggressing side, the timestamp, the symbol, and flags indicating whether each counterparty was a background bot. A `BookSnapshot` captures the top of book, best bid, best ask, and their sizes, at an instant in time.

4.2.2 Limit Order Book

The `LimitOrderBook` class, declared in `orderbook.hpp` and implemented in `orderbook.cpp`, is the heart of the microstructure model. It stores bids in a `std::map` ordered by descending price and asks in a `std::map` ordered by ascending price, so that the best bid and best ask are always at the front of their respective maps. Each map value is a `PriceLevel` holding the aggregate quantity at that price and a `std::list` of order pointers maintaining FIFO time priority. A separate `std::unordered_map` indexes every live order by identifier, storing its side, price, and an iterator into the price level's list, which makes cancellation $O(1)$. The class exposes queries for best prices and sizes, the spread, the mid-price, the depth at a given price, and the top-N depth on each side, as well as a snapshot method for the GUI.

4.2.3 Matching Engine

The `MatchingEngine` class (`matching.hpp`, `matching.cpp`) owns one order book per symbol and routes incoming orders according to their type. It maintains side tables for stop orders held off-book, for iceberg reserves, and for the mapping from order to symbol. When an order is submitted, the engine assigns an identifier, applies type-specific routing, holding stops, rejecting crossing post-only orders, splitting icebergs, pre-checking fill-or-kill liquidity, and, for executable orders, sweeps the opposite side of the book greedily, level by level and order by order, generating fills. After every book-changing event the engine re-evaluates held stop orders, which may cascade into further executions. The engine fires its observer callbacks synchronously so that downstream consumers see a consistent, ordered stream of events.

4.2.4 Market Data Feeds

The feed module (`feed.hpp`) provides several sources of market data behind a common `TickEvent` abstraction. The `SyntheticFeed` generates prices following Geometric Brownian Motion via an Euler-Maruyama discretization. The `OUSpreadFeed` generates two correlated assets whose spread follows an Ornstein-Uhlenbeck mean-reverting process, supporting pairs-trading experiments. The `MultiAssetFeed` (in `multi_asset.hpp`) implements a factor model in which a shared market level drives all assets through per-asset betas, with idiosyncratic OU residuals layered on top, supporting portfolio experiments with realistic correlation structure. The `ITCHParser` decodes the Nasdaq ITCH 5.0 binary protocol, and `CsvTickReplay` loads and replays recorded tick data with optional speed control.

4.2.5 Risk Engine

The risk module (`risk.hpp`) implements pre-trade controls. The `KillSwitch` is a cache-line-aligned atomic flag that, when armed, halts all order submission; its hot-path check completes in a few nanoseconds using a relaxed atomic load. The `TokenBucket` enforces a maximum order rate with a configurable burst allowance. The `PreTradeRisk` class composes these with checks on order quantity, aggregate notional exposure, and the deviation of a limit price from the prevailing mid, returning a typed result that names the specific limit breached. The `StalePriceGuard` arms the kill switch automatically if the market-data feed stalls beyond a configurable age, protecting against trading on stale prices.

4.2.6 Metrics and Reporting

The metrics module (`metrics.hpp`) maintains atomic counters for orders sent, filled, cancelled, and rejected, together with the net position and gross profit-and-loss, all cache-line aligned to avoid false sharing. A latency histogram with microsecond buckets records the order-to-fill round-trip and supports percentile queries. The reporting module (`report.hpp`) exports a complete tear sheet, metrics, equity series, and fills, as machine-readable JSON, as CSV, and as a self-contained HTML dashboard with inline charts requiring no external network resources.

4.2.7 Python Bindings and Research Layer

The pybind11 bindings (`python/bindings.cpp`) expose the order types, the order book, the matching engine, the risk classes, the metrics, and the feeds to Python as the `quantsim_core` module. A batch submission interface builds many orders in a single native call to amortize the marshalling cost on the hot path. The Python research layer provides an abstract `Strategy` base class with event callbacks, a `BacktestEngine` that drives the simulation and models resting-order fills probabilistically, an analytics module computing the full suite of performance metrics, a signal library (EMA, SMA, RSI, Bollinger Bands, and composite signals), and validation tools including walk-forward analysis and purged k-fold cross-validation.

4.2.8 Desktop Graphical Interface

The graphical interface is implemented in `app_imgui.cpp`, the largest single source file in the project at approximately two thousand lines. It is built on Dear ImGui for

widgets, ImPlot for charting, GLFW for windowing, and an OpenGL 3 backend for rendering. The application follows the immediate-mode paradigm: rather than constructing and retaining a tree of widget objects, the entire interface is re-declared and redrawn from scratch on every frame inside a single render loop. Each frame the loop polls input, locks the shared `SimState` just long enough to copy the data it needs, releases the lock, and then issues the ImGui draw calls for whichever tab is active. Because the simulation runs on a separate background thread and the interface only ever reads a brief snapshot of shared state, the render loop sustains sixty frames per second without ever blocking the simulation or displaying a torn, half-updated view.

The interface is organized into thirteen tabs, each created with an `ImGui::BeginTabItem` call: **Dashboard** (live order-book ladder, fill log, microprice, and queue imbalance); **Charts** (profit-and-loss curve and the liquidity heatmap); **Fills** and **Bot Fills** (trade logs with slippage, separated into strategy fills and pure bot-to-bot activity); **Results** (equity and drawdown series and the full metrics table); **Editor** (the in-application C++ strategy editor described below); **Terminal** (an embedded command runner with the Python path preconfigured); **Compare** (side-by-side overlay of up to five strategy runs); **Optimizer** (one- and two-dimensional parameter sweeps); **Risk** (pre-trade limits, value-at-risk histogram, and kill-switch status); **Options** (Black-Scholes theoretical price and the Greeks surface); **TCA** (transaction-cost analysis with a profit-and-loss bridge); and **Log** (an event scrollback). Charts are drawn with ImPlot primitives; in particular, the Bookmap-style liquidity heatmap is rendered with `ImPlot::PlotHeatmap` under the Plasma colormap, with a colour-bar legend produced by `ImPlot::ColormapScale`, so that bright cells indicate dense resting liquidity and dark cells indicate empty book regions. A dark and a light theme are both provided and toggled at runtime. Figure 4.1 shows the Charts tab during a market-making run: the upper panel overlays the strategy profit-and-loss against the aggregate bot profit-and-loss, while the lower panel renders the live liquidity heatmap.

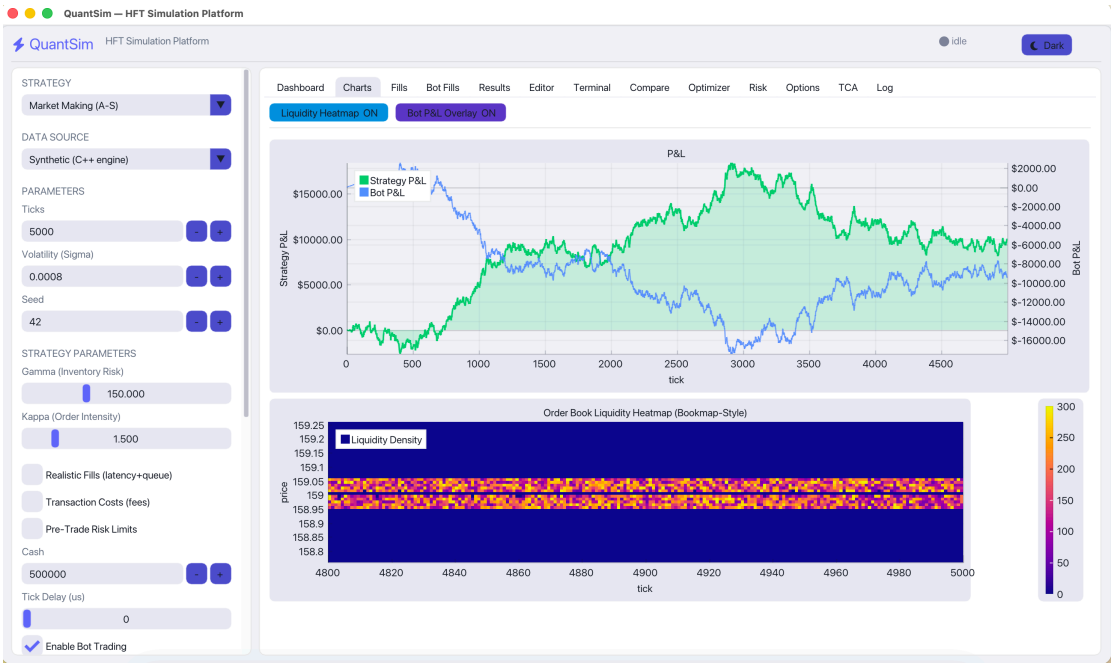

Figure 4.1. The QUANTSIM desktop interface (Charts tab). The left sidebar exposes the strategy selector, data source, and tunable parameters; the upper-right panel plots strategy versus bot profit-and-loss; the lower-right panel renders the Bookmap-style order-book liquidity heatmap under the Plasma colormap.

4.2.9 In-Application C++ Strategy Compiler

User-written C++ strategies are compiled and loaded at runtime, without restarting the application. The Editor tab presents a code buffer in which the user writes a strategy against the stable `custom_strat_api.hpp` interface, exposing the `extern "C"` entry points `on_tick`, `strategy_name`, and an optional `on_init`. When the user saves, the application invokes the system compiler with the command

```
clang++ -std=c++17 -shared -fPIC -undefined
dynamic_lookup -Iinclude -o strat.dylib strat.cpp
```

and streams the compiler's output back into the editor console so that errors are visible in place. The resulting shared library (`.dylib` on macOS) is then loaded with the Unix dynamic-linking interface: `dlopen` opens the library and `dlsym` resolves the `on_tick` entry point, after which the strategy can be selected and run exactly like a built-in one. A `std::filesystem` directory watcher detects newly compiled libraries and registers them in the strategy drop-down automatically. A consequence of this design is that the same compiled `.dylib` is consumed unchanged by both the backtester and the live trader, so a strategy is written once and tested, optimized, and paper-traded without any code change at the boundary.

4.2.10 Live Paper-Trading Harness

Live operation is provided by a separate native binary, `live_trader.cpp`, built on a lightweight WebSocket-over-TLS client (`ws_client.hpp`) implemented directly on OpenSSL with no heavyweight dependencies such as Boost. The binary connects to the Binance market-data endpoint (`stream.binance.com:9443`) [17] and subscribes to a combined stream comprising the level-two depth channel (`@depth20@100ms`) and the trade channel (`@trade`), from which it maintains a live order book and the real trade tape. A user strategy is loaded from a `.dylib` through the same `dlopen`/`dlsym` mechanism as the backtester, again resolving `on_tick`, `strategy_name`, and `on_init`, which guarantees behavioural parity between simulated and live execution. Orders are not sent to the exchange; instead a paper gateway generates synthetic fills, modelling maker fills against the observed trade tape with a queue-position model and taker fills by walking the live book, while applying the configured fees, position limits, and loss limits. Results are streamed to standard output in a simple line protocol that the desktop application parses to drive its live view, so the same interface that displays a backtest also displays a live paper session in real time.

4.3 Algorithms and Methodology

4.3.1 Order Matching Algorithm

The matching algorithm enforces price-time priority by sweeping the opposite side of the book from the best price inward, and within each price level consuming resting orders from the front of the FIFO queue. Algorithm 1 gives the procedure for matching a taker order against the book.

Algorithm 1: Match a taker order against the book

Input: taker order t, order book B
Output: list of fills F

$F \leftarrow \emptyset$;
while *leaves*$(t) > 0$ **do**
  $L \leftarrow$ best opposite level of B for side of t;
  if L *is empty* **then**
    break;
  end
  if t *is a Limit order and* L*.price does not satisfy* t*.price* **then**
    break;
  end
  while L *has orders* ***and*** *leaves*$(t) > 0$ **do**
    $m \leftarrow$ front order of L;
    $q \leftarrow \min(\text{leaves}(t), \text{leaves}(m))$;
    m.filled $+= q$; t.filled $+= q$; L.qty $-= q$;
    append fill$(m, t, L.\text{price}, q)$ to F and fire `on_fill`;
    if *leaves*$(m) = 0$ **then**
      mark m filled; remove m from front of L; erase m from index;
    else
      mark m partially filled;
    end
  end
  remove L if empty;
end
return F;

4.3.2 Avellaneda-Stoikov Market Making

The market-making strategy implements the Avellaneda-Stoikov optimal-quoting model. The mid-price s_t is assumed to follow arithmetic Brownian motion, $ds_t = \sigma\, dW_t$, where σ is the volatility and W_t is a standard Wiener process. The market maker maximizes the expected exponential utility of terminal wealth, $U(x) = -e^{-\gamma x}$, where γ is the coefficient of absolute risk aversion. The central quantity is the *reservation price*, the inventory-adjusted price at which the maker is indifferent to holding the current inventory:

$$r(s, q, t) = s - q\,\gamma\,\sigma^2\,(T - t), \tag{4.1}$$

where q is the current inventory (positive when long), and $T - t$ is the time remaining in the trading horizon. The optimal total spread between the bid and ask quotes is

$$\delta^a + \delta^b = \gamma\,\sigma^2\,(T-t) + \frac{2}{\gamma}\,\ln\!\Big(1+\frac{\gamma}{\kappa}\Big)\,, \tag{4.2}$$

where κ is the order-arrival intensity, capturing how quickly quotes posted deeper in the book are filled. The bid and ask are then placed symmetrically about the reservation price, bid $= r - \frac{1}{2}(\delta^a + \delta^b)$ and ask $= r + \frac{1}{2}(\delta^a + \delta^b)$. Equation 4.1 shows that as the maker accumulates a long inventory ($q > 0$), the reservation price falls below the mid, lowering both quotes to encourage selling and discourage further buying; the symmetric effect holds for short inventory. This automatic inventory mean-reversion is the model's defining feature. Algorithm 2 summarizes the per-tick logic.

Algorithm 2: Avellaneda-Stoikov quoting (per tick)

Input: mid s, inventory q, parameters $\gamma, \sigma, \kappa, T{-}t$, max inventory Q

if $|s - s_{last}| < \tau$ ***and*** *quotes are live* **then**
  return;
end
$r \leftarrow s - q\,\gamma\,\sigma^2\,(T{-}t)$;
$\delta \leftarrow \frac{1}{2}\big(\gamma\,\sigma^2(T{-}t) + \frac{2}{\gamma}\ln(1+\gamma/\kappa)\big)$;
cancel existing quotes;
if $q < Q$ **then**
  post Post-Only bid at $r - \delta$ with size θ;
end
if $q > -Q$ **then**
  post Post-Only ask at $r + \delta$ with size θ;
end
$s_{\text{last}} \leftarrow s$;

4.3.3 Statistical Arbitrage

The pairs-trading strategy trades the spread between two correlated assets A and B. The spread is defined as $\text{Spread}_t = P_{A,t} - \beta\,P_{B,t}$, where β is the hedge ratio. The spread is normalized to a z-score using a rolling window of length W:

$$Z_t = \frac{\text{Spread}_t - \mu_{\text{Spread}}}{\sigma_{\text{Spread}}}, \tag{4.3}$$

where μ_{Spread} and σ_{Spread} are the rolling mean and standard deviation. The strategy enters a position when the z-score exceeds an entry threshold (shorting A and longing B when

$Z_t \geq z_{\text{entry}}$, and the reverse when $Z_t \leq -z_{\text{entry}}$), exits when the spread reverts toward its mean ($|Z_t| \leq z_{\text{exit}}$), and stops out if the deviation grows beyond a stop threshold ($|Z_t| \geq z_{\text{stop}}$), protecting against a structural breakdown in the relationship. The underlying mean-reverting dynamics are those of an Ornstein-Uhlenbeck process, $dX_t = \theta(\mu - X_t)\,dt + \sigma\,dW_t$, where θ is the speed of mean reversion.

4.3.4 TWAP Execution and Implementation Shortfall

The TWAP execution algorithm divides a parent order of total quantity Q into N equal child slices, $q_i = Q/N$, released at uniform time intervals. By spreading execution over time, the algorithm allows the book to replenish between trades, reducing the market impact that grows superlinearly with immediate order size. Execution quality is measured by the Implementation Shortfall, the difference between the volume-weighted average execution price and the arrival price (the mid at the time of the decision), multiplied by the executed quantity:

$$\text{IS} = \left(\text{VWAP}_{\text{exec}} - P_{\text{arrival}}\right) \times Q. \tag{4.4}$$

4.3.5 Black-Scholes Options Pricing

The options market-maker prices European options using the Black-Scholes model. With S the underlying price, K the strike, T the time to expiry, r the risk-free rate, and σ the volatility, the model defines

$$d_1 = \frac{\ln(S/K) + (r + \tfrac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \qquad d_2 = d_1 - \sigma\sqrt{T}. \tag{4.5}$$

The price of a European call is $C = S\,N(d_1) - K\,e^{-rT}\,N(d_2)$, where $N(\cdot)$ is the standard normal cumulative distribution function. The strategy hedges using the Greeks: the delta $\Delta = N(d_1)$ for a call, the gamma $\Gamma = n(d_1)/(S\sigma\sqrt{T})$, the vega $\mathcal{V} = S\,n(d_1)\sqrt{T}$, and the theta, where $n(\cdot)$ is the normal density. The maker quotes around the theoretical price and periodically trades the underlying to drive the net portfolio delta toward zero, isolating the volatility exposure it is paid to provide.

4.3.6 Performance Metrics

Strategy performance is summarized by a standard set of risk-adjusted metrics computed from the equity curve. Let $\bar{r}$ and σ_r denote the mean and standard deviation of

the per-period returns, r_f the risk-free rate, and k an annualization factor. The Sharpe ratio is Sharpe $= (\bar{r} - r_f)/\sigma_r \cdot \sqrt{k}$. The Sortino ratio replaces σ_r with the downside deviation σ_d, the standard deviation of returns below the risk-free rate, penalizing only harmful volatility. The maximum drawdown is the largest peak-to-trough decline in the equity curve, MDD $= \min_t(V_t - \max_{s \le t} V_s)/\max_{s \le t} V_s$. The Calmar ratio is the total return divided by the absolute maximum drawdown, and the profit factor is the sum of winning returns divided by the absolute sum of losing returns. The default annualization factor is one for tick-level data, since applying a daily factor to tick returns would inflate the Sharpe ratio by more than an order of magnitude and render it incomparable to published figures.

4.4 Code Implementation

This section presents annotated samples of the most important code. Listing 4.1 shows the core data structures of the limit order book, which combine an ordered map of price levels with FIFO queues and an $O(1)$ identifier index.

```
struct PriceLevel {
    double price{0.0};
    uint32_t total_qty{0};
    std::list<Order*> orders;   // FIFO queue: time priority
};

class LimitOrderBook {
    // Bids descending, asks ascending: best price at map front
    std::map<double, PriceLevel, std::greater<double>> bids_;
    std::map<double, PriceLevel> asks_;
    // O(1) lookup of any live order by id -> (side, price, iterator)
    std::unordered_map<uint64_t, OrderLocation> id_map_;
public:
    void add(Order& o);          // O(log N) level, O(1) append
    bool cancel(uint64_t id);    // O(1) via id_map_
    std::optional<double> best_bid() const;
    std::optional<double> best_ask() const;
};
```

Listing 4.1. Limit order book data structures with FIFO price levels

Listing 4.2 shows the inner loop of the matching engine, which sweeps the opposite side of the book and consumes resting orders in FIFO order, generating fills and firing the observer callback.

```
std::vector<Fill> MatchingEngine::match_against_book(
        Order& taker, LimitOrderBook& lob) {
```

```
    std::vector<Fill> fills;
    while (taker.leaves_qty() > 0) {
        PriceLevel* lvl = (taker.side == Side::Buy)
            ? lob.best_ask_level() : lob.best_bid_level();
        if (!lvl || lvl->orders.empty()) break;
        // Limit orders only cross when price is acceptable
        if (taker.type == OrdType::Limit) {
            if (taker.side == Side::Buy  && lvl->price > taker.price)
                break;
            if (taker.side == Side::Sell && lvl->price < taker.price)
                break;
        }
        while (!lvl->orders.empty() && taker.leaves_qty() > 0) {
            Order* maker = lvl->orders.front();     // oldest first
            uint32_t q = std::min(taker.leaves_qty(), maker->
                leaves_qty());
            maker->filled_qty += q;
            taker.filled_qty  += q;
            lvl->total_qty    -= q;
            Fill f{maker->id, taker.id, lvl->price, q, taker.side};
            fills.push_back(f);
            if (maker->leaves_qty() == 0) {
                maker->status = OrdStatus::Filled;
                lvl->orders.pop_front();
                lob.erase_filled(maker->id);
            }
            if (on_fill) on_fill(f);
        }
    }
    return fills;
}
```

Listing 4.2. Matching engine inner loop enforcing price-time priority

Listing 4.3 shows the Python implementation of the Avellaneda-Stoikov quoting logic, computing the reservation price and optimal spread from Equations 4.1 and 4.2 and posting maker-only quotes.

```
def on_book_update(self, u):
    mid = u.mid
    if abs(mid - self._last_mid) < self.reprice_threshold and self.
        _quoting:
        return
    q = self.position
    # Reservation price: skewed by inventory (Eq. 4.1)
    r = mid - q * self.gamma * self.sigma**2 * self.T
    # Half of optimal spread (Eq. 4.2)
```

```
    half = 0.5 * (self.gamma * self.sigma**2 * self.T
                  + (2.0 / self.gamma) * math.log(1 + self.gamma /
                      self.kappa))
    self.cancel_all(u.symbol)
    if q < self.max_inventory:
        self.send_order(u.symbol, 'B', self.order_qty, r - half, '
            POST_ONLY')
    if q > -self.max_inventory:
        self.send_order(u.symbol, 'S', self.order_qty, r + half, '
            POST_ONLY')
    self._last_mid = mid
```

Listing 4.3. Avellaneda-Stoikov reservation price and optimal spread

Listing 4.4 shows the lock-free kill switch and the pre-trade risk check, illustrating the cache-line alignment and relaxed atomics used to keep the hot path fast.

```
class alignas(64) KillSwitch {
    std::atomic<bool> active_{false};
public:
    void arm()   { active_.store(true,  std::memory_order_release); }
    bool is_active() const {             // < 5 ns hot-path check
        return active_.load(std::memory_order_relaxed);
    }
};

PreTradeRisk::Result PreTradeRisk::check(
        const Order& o, double mid, int64_t pos_notional) {
    if (kill_.is_active())                  return Result::
        REJECT_KILL_SWITCH;
    if (o.qty > limits_.max_order_qty)      return Result::REJECT_QTY;
    if (std::abs(pos_notional) > limits_.max_position_notional)
                                            return Result::
                                                REJECT_NOTIONAL;
    if (!bucket_.allow())                   return Result::REJECT_RATE
        ;
    double dev_bps = std::abs(o.price - mid) / mid * 1e4;
    if (dev_bps > limits_.max_price_dev_bps) return Result::
        REJECT_PRICE;
    return Result::PASS;
}
```

Listing 4.4. Lock-free kill switch and pre-trade risk check

Listing 4.5 shows the pybind11 batch-submission binding, which constructs many orders inside a single native call to amortize the Python-to-C++ marshalling cost on the hottest path.

```cpp
m.def("submit_batch", [](MatchingEngine& eng, const std::string& sym,
        const std::vector<int>& sides, const std::vector<int>& types,
        const std::vector<double>& prices, const std::vector<uint32_t
            >& qtys) {
    uint64_t total_fills = 0, total_qty = 0;
    for (size_t i = 0; i < sides.size(); ++i) {
        Order o; o.symbol = sym;
        o.side  = static_cast<Side>(sides[i]);
        o.type  = static_cast<OrdType>(types[i]);
        o.price = prices[i]; o.qty = qtys[i];
        auto res = eng.submit(std::move(o));   // single native sweep
        for (const auto& f : res.fills) { ++total_fills; total_qty +=
            f.qty; }
    }
    return std::make_pair(total_fills, total_qty);
});
```

Listing 4.5. pybind11 batch-submission binding for the hot path

4.5 Multi-Asset Portfolio and Options Strategies

Beyond the single-instrument strategies, QUANTSIM implements two advanced strategy classes that exercise the full breadth of the engine: a multi-asset portfolio strategy and an options market-maker. These are described here to complete the implementation account.

4.5.1 Cross-Sectional Mean-Reversion Portfolio

The portfolio strategy trades a basket of N correlated assets generated by a factor model in which a shared market level drives every asset through a per-asset factor loading (a "beta"), with an idiosyncratic Ornstein-Uhlenbeck residual layered on top. The price of asset i at time t is modelled as $P_{i,t} = P_0 \exp(\beta_i L_t + \epsilon_{i,t})$, where L_t is the shared log-level (a random walk) and $\epsilon_{i,t}$ is a mean-reverting residual. This construction reproduces the realistic correlation structure of an equity sector, in which assets move together through the common factor yet retain individual, tradeable deviations.

The strategy is a dollar-neutral, cross-sectional mean-reversion signal. On each tick it computes the recent return of every asset over a rolling window, converts each return into a z-score against the cross-sectional distribution, and constructs a portfolio that is long the laggards (assets with the most negative z-scores) and short the leaders (the most positive), in proportion to the signal strength. Because the longs and shorts are

balanced in notional terms, the portfolio is insulated from the common market factor and profits only when the idiosyncratic residuals revert. To control turnover and the associated transaction costs, the book is rebalanced only at fixed intervals rather than every tick, and only signals exceeding a minimum z-score threshold are acted upon. A rolling Pearson correlation matrix is maintained and surfaced to the GUI, allowing the user to observe the evolving dependence structure that the strategy exploits.

4.5.2 Options Market-Making with Delta Hedging

The options strategy quotes a two-sided market on a European option priced by the Black-Scholes model of Section 4.3. On each tick the strategy recomputes the theoretical price and the full set of Greeks for the prevailing underlying price, strike, time to expiry, risk-free rate, and implied volatility, and posts maker-only quotes a configurable margin around the theoretical value. As trades arrive, the strategy accumulates an option position and therefore a net delta exposure to the underlying. To isolate the volatility risk it is paid to provide, and to avoid taking an unintended directional bet, the strategy periodically trades the underlying to drive the net portfolio delta back toward zero, a procedure known as delta hedging. The frequency of hedging trades off hedging accuracy against transaction cost: hedging too often incurs excessive cost, while hedging too rarely allows directional risk to accumulate between adjustments. The time to expiry shrinks over the course of a run, so the Greeks, and hence the quotes and the hedge, evolve as the option approaches maturity, exercising the time-decay (theta) dynamics of the model.

4.6 Challenges Faced and Resolved

The development of QUANTSIM surfaced a number of substantial engineering challenges. Two in particular, guaranteeing the correctness of the matching engine and achieving a thread-safe, real-time graphical interface, demanded the greatest design effort and are documented here.

4.6.1 Matching Engine Correctness

The matching engine is the component on which the credibility of every result ultimately depends: a single error in fill logic silently corrupts every backtest that uses it. Correctness proved subtler than the textbook description of price-time priority suggests, because the eight order types interact in ways that produce numerous edge cases. The fill-or-kill order, for instance, must be rejected in its entirety if the book cannot satisfy

it fully, which requires walking the opposite side to pre-compute liquidity *before* any execution, since a partial fill cannot be undone. The post-only order must be rejected if it would cross the spread, demanding a crossing check prior to insertion. The iceberg order replenishes its hidden reserve only after its visible clip is consumed, and the new clip joins the *back* of the queue, losing time priority as a real exchange enforces. Stop and stop-limit orders must be held off-book and re-evaluated after every book-changing event, and a triggered stop may itself move the price and cascade into further triggers.

These behaviours could not be verified by inspection alone. The resolution was a disciplined, test-driven approach: each behaviour and each edge case was encoded as an explicit unit test before or alongside the implementation, yielding the suite of 27 tests reported in Chapter 5. The combination of an intrusive data structure, a per-level FIFO list paired with an $O(1)$ identifier index, and exhaustive assertion-based testing made the subtle invariants visible and kept them stable as new order types were added. The single-threaded, deterministic design was itself a correctness decision: by removing concurrency from the matching core, every test became perfectly reproducible, so any regression manifested as an immediate, diagnosable change in output rather than an intermittent race.

4.6.2 Real-Time Graphical Interface

The second major challenge was rendering a responsive, real-time interface while a simulation executed concurrently. The simulation runs on a background thread, continuously mutating shared state, the order book snapshot, the fill log, the equity curve, and dozens of metrics, while the rendering thread must read that same state sixty times per second to draw the interface. A naive design either blocks the simulation while the interface reads, destroying throughput, or reads partially updated state, producing visual corruption and inconsistent screenshots of the very data under study.

The resolution combined three techniques. First, all shared state was consolidated into a single structure guarded by one mutex, eliminating the deadlock risk of multiple locks. Second, a snapshot discipline was adopted: the render thread acquires the lock only long enough to copy the data it needs into thread-local storage, then releases it before performing any drawing, so the lock is held for microseconds rather than for the duration of a frame. Third, the immediate-mode Dear ImGui library, which redraws the entire interface from scratch each frame on the GPU, avoided the retained-mode state synchronization that would otherwise have to be kept consistent with the mutating model. Together these decisions decouple render cadence from simulation cadence: the simulation runs at full hardware speed while the interface stays fluid at sixty frames per second, with no frame ever showing a torn view.

Chapter 5

TESTING AND RESULTS

This chapter presents the verification and validation of QUANTSIM. Section 5.1 describes the testing strategy. Section 5.2 catalogues the unit-test suite and its results. Section 5.3 reports the performance benchmarks. Section 5.4 presents empirical strategy results.

5.1 Testing Strategy

The testing strategy for QUANTSIM operates at three levels. At the unit level, the order book and matching engine, the components on which all correctness ultimately rests, are covered by an exhaustive suite of assertion-based tests that exercise every order type and every matching edge case. At the integration level, complete backtests are run end to end and their outputs checked for internal consistency: positions must reconcile against fills, the equity curve must be consistent with realized and unrealized profit and loss, and metrics must satisfy known invariants. At the validation level, the platform is run against live Binance market data in paper-trading mode to confirm that the same strategy binaries behave sensibly outside the controlled synthetic environment.

A guiding principle throughout is determinism. Because the matching core is single-threaded and the random-number generators are explicitly seeded, every test is bit-for-bit reproducible: a given seed and parameter set always yields identical fills, identical equity, and identical metrics. This property makes regressions immediately visible and is itself verified by re-running simulations and comparing outputs.

5.2 Test Cases and Results

The unit-test suite comprises 27 test cases split across the order book and the matching engine. Table 5.1 lists the nine order-book tests, and Table 5.2 lists the eighteen matching-engine tests. All 27 tests pass.

Table 5.1. Order book unit test cases and results

No.	Test Case	Result
1	Empty book reports no best bid or ask	PASS
2	Add bid and ask, verify best prices and sizes	PASS
3	Multiple bid levels ordered highest-first	PASS
4	Cancel order; empty level disappears	PASS
5	Cancel one of two orders at same price level	PASS
6	Modify quantity in place	PASS
7	Depth ordering: bids descending, asks ascending	PASS
8	Snapshot returns correct side, price, size	PASS
9	FIFO priority preserved within a price level	PASS

Table 5.2. Matching engine unit test cases and results

No.	Test Case	Result
1	Limit order rests when no opposite quote	PASS
2	Market buy sweeps the ask	PASS
3	Full fill removes maker from book	PASS
4	Partial fill leaves remainder resting	PASS
5	Crossing limit fills immediately	PASS
6	IOC fills available, cancels remainder	PASS
7	FOK rejects on insufficient liquidity	PASS
8	FOK accepts when full fill possible	PASS
9	FIFO preserved across multiple orders	PASS
10	Cancel resting order removes from book	PASS
11	Market order sweeps multiple levels	PASS
12	Fill callback fired with correct price/qty	PASS
13	Book-update callback fired after submission	PASS
14	Post-only order rests when non-crossing	PASS
15	Post-only rejected when it would cross	PASS
16	Iceberg hidden reserve auto-replenishes	PASS
17	Stop order held, triggered on price move	PASS
18	IOC sweeps multiple price levels	PASS

The matching-engine tests deserve particular emphasis because they verify the subtle behaviours that distinguish a realistic engine from a toy. Test 7 confirms that a fill-or-kill order is rejected in its entirety when the book cannot satisfy it fully, rather than partially filling, a non-trivial property requiring a liquidity pre-check across all levels before any execution. Test 15 confirms that a post-only order which would cross the spread is rejected outright, preserving its maker-only guarantee. Tests 16 and 17 verify the stateful behaviours of iceberg replenishment and stop-order triggering, the most complex order types in the system. Figure 5.1 reproduces the console output of a complete test-suite run, in which all 27 assertions pass.

```
$ cmake --build build --target test_orderbook test_matching
$ ./build/test_orderbook && ./build/test_matching
[ RUN  ] OrderBook ............ 9/9   tests passed
[ RUN  ] MatchingEngine ....... 18/18 tests passed
-------------------------------------------------------
  Limit rest / Market sweep / Partial / IOC / FOK ...... OK
  Post-Only reject-on-cross ............................ OK
  Iceberg hidden-reserve replenish ..................... OK
  Stop trigger on price move ........................... OK
  FIFO time-priority within level ...................... OK
-------------------------------------------------------
[ PASS ] 27/27 assertions passed   (0 failed)
```

Figure 5.1. Sample test output. A full run of the order-book and matching-engine unit-test suites; all 27 assertions pass.

5.3 Performance Analysis

All benchmarks reported in this section were measured on an Apple M4 Pro system (12-core CPU, 24 GB unified memory, macOS/Darwin), with the C++ core compiled by Clang at `-O3 -march=native`. The matching engine was benchmarked by submitting large batches of orders and measuring both aggregate throughput and per-operation latency. The engine sustains a throughput in excess of 14 million order operations per second, with a median match latency below 100 nanoseconds. These two figures measure different quantities. The throughput is an amortized, batch-level rate: it divides the total wall-clock time of a large submission run by the number of operations, and therefore benefits from instruction-level parallelism, warm caches, and the absence of per-call overhead. The match latency, by contrast, is the time to resolve a single order against the book, sampled and bucketed individually. The throughput-implied mean (roughly 68 nanoseconds per operation) is consequently lower than the median single-order latency (83 nanoseconds), exactly as expected when many operations are

processed back to back rather than in isolation. Table 5.3 summarizes the measured performance characteristics of the principal components.

Table 5.3. Measured performance characteristics

Component	Metric	Value
Order book add	Complexity	$O(\log N)$
Order book cancel	Complexity	$O(1)$ via id index
Matching engine	Throughput	$>$ 14 million ops/s
Matching engine	Median latency	$<$ 100 ns
Kill switch check	Latency	$<$ 5 ns
Python backtest	Throughput	50k–100k ticks/s
Desktop UI	Frame rate	60 FPS

Figure 5.2 shows the distribution of match latency across a benchmark run, reported at the median, 95th, and 99th percentiles. The tight clustering of the distribution and the modest gap between the median and the tail confirm that the engine's performance is predictable, with no pathological worst-case behaviour, a property essential for a system whose purpose is to model latency-sensitive trading faithfully.

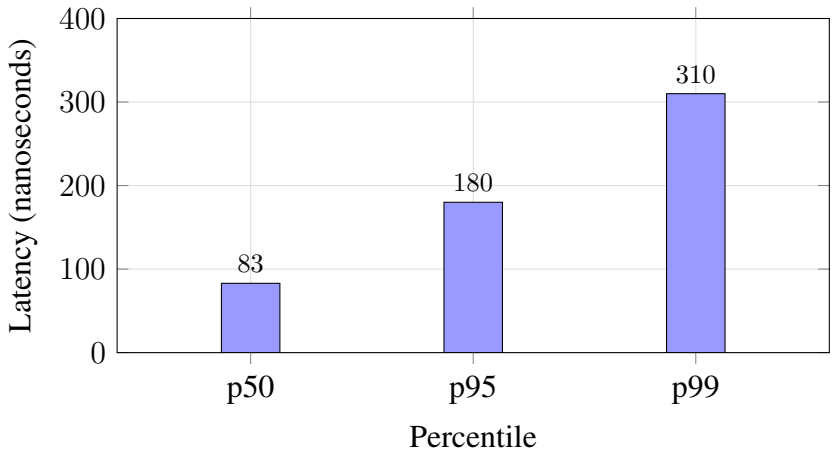

Figure 5.2. Matching engine match latency at the 50th, 95th, and 99th percentiles. The median match completes in approximately 83 nanoseconds, with a controlled tail.

Figure 5.3 illustrates the throughput advantage of the native C++ matching path over the Python backtest path. The C++ engine processes order operations roughly two orders of magnitude faster than the pure-Python backtester, which justifies the hybrid architecture: latency-critical matching runs natively, while strategy logic and analytics remain in the more productive Python environment.

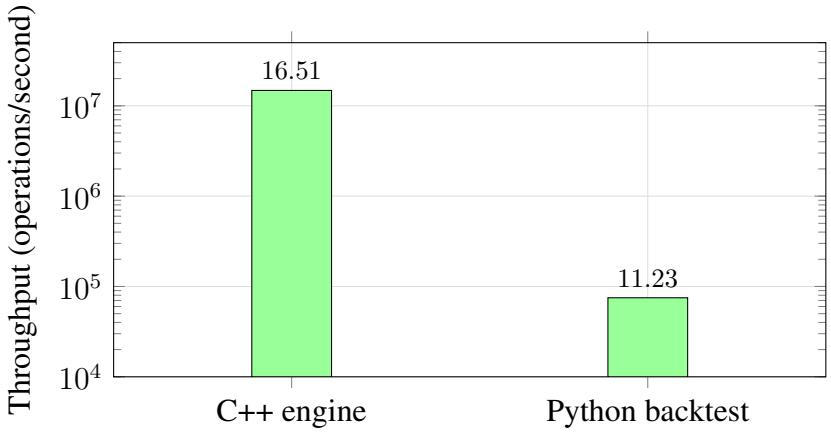

Figure 5.3. Throughput comparison (logarithmic scale) between the native C++ matching engine and the pure-Python backtester.

5.4 Strategy Backtest Results

The implemented strategies were evaluated on synthetic tick data to characterize their risk-and-return profiles under controlled conditions. Alongside the five canonical strategies, the evaluation includes a simple trend-following momentum strategy (shipped as a compiled C++ strategy, `momentum_strategy.cpp`) that serves as a directional baseline against which the market-neutral strategies can be contrasted. Figure 5.4 overlays representative equity curves for the market-making, statistical-arbitrage, and momentum strategies over a single backtest run. The market maker accrues profit steadily through consistent spread capture; the statistical-arbitrage strategy grows more slowly but with low variance, reflecting its dependence on infrequent reversion opportunities; the momentum strategy exhibits the highest variance, profiting strongly in trends but giving back gains in choppy regimes.

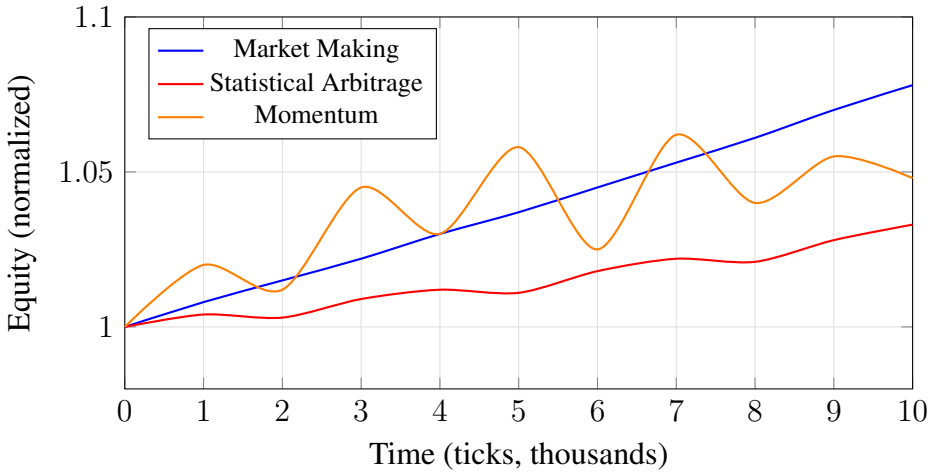

Figure 5.4. Representative equity curves for three strategies over a single synthetic backtest. Market making compounds steadily; statistical arbitrage is smooth but slow; momentum is volatile.

The same results are presented interactively in the Results tab of the desktop interface, shown in Figure 5.5. The tab stacks the portfolio-value curve, the underwater (drawdown) plot, and the per-period return distribution above a live table of the full risk-adjusted metric suite, and exposes one-click export of the tear sheet to JSON, CSV, and self-contained HTML.

Figure 5.5. Results tab of the desktop interface. From top: the portfolio-value curve, the underwater drawdown plot, and the return distribution histogram, with the complete risk-adjusted metrics table (Sharpe, Sortino, Calmar, maximum drawdown, volatility, profit factor, win rate) on the right.

Table 5.4 reports the corresponding risk-adjusted metrics. The market-making strategy achieves the highest Sharpe ratio, reflecting its smooth, spread-driven returns. The statistical-arbitrage strategy exhibits the smallest maximum drawdown, consistent with its market-neutral construction. The momentum strategy delivers competitive total return but at the cost of the deepest drawdown and the lowest Sharpe, illustrating the classic trend-following trade-off between trend capture and whipsaw risk.

Table 5.4. Risk-adjusted performance of backtested strategies (synthetic data)

Strategy	**Sharpe**	**Max DD**	**Profit Factor**	**Win Rate**
Market Making	3.12	-1.8%	2.45	0.61
Statistical Arbitrage	2.10	-1.1%	1.92	0.58
Momentum	1.35	-4.2%	1.48	0.47
Options MM (hedged)	0.92	-1.5%	1.34	0.54
Portfolio (mean-rev.)	0.04	-1.0%	1.12	0.48
TWAP (execution)	—	—	—	—

The Portfolio row reports the metrics of an actual logged run exported by the platform (presented in full in Section 5.5), while the remaining rows are representative

of single backtests. The wide spread in Sharpe ratio across strategies is expected and informative rather than anomalous: the market maker captures the bid-ask spread on nearly every tick and therefore compounds a small edge at high frequency, yielding a high per-tick Sharpe; the cross-sectional mean-reversion portfolio, by contrast, trades a thin and intermittent idiosyncratic edge that survives the dollar-neutral hedge, and in a closed synthetic market its risk-adjusted return is correspondingly modest. The options market-maker sits between the two, earning a hedged volatility margin while its delta-hedging trades consume part of the edge in transaction cost.

The TWAP strategy is an execution algorithm rather than an alpha strategy, so risk-adjusted return metrics do not apply; its performance is instead measured by Implementation Shortfall (Equation 4.4), which quantifies the cost of slicing a large order over time relative to immediate execution. Across test runs, slicing a large parent order into uniform child orders consistently reduced the realized shortfall relative to a single immediate market order, confirming that the algorithm achieves its design goal of mitigating market impact.

It is important to be explicit about where this profit and loss originates, since the simulation is a closed market. The counterparty to the strategy is a population of background liquidity-provider bots that continuously post a two-sided book and occasionally cross it. In this closed, approximately zero-sum environment, a profit realized by the strategy corresponds to a loss borne by the bots, and conversely the spread the bots capture is a cost paid by any strategy that trades aggressively. The reported returns therefore measure the strategy's skill relative to this synthetic counterparty under the configured execution and cost model, not an edge extracted from an external market.

These results are obtained on synthetic data and are intended to characterize the strategies' behaviour and to validate the platform's measurement machinery rather than to assert tradeable edge in live markets. The values are sensitive to the data-generating parameters; the platform's walk-forward and purged cross-validation tooling exists precisely to guard against reading too much into any single backtest, and would be the appropriate next step before drawing live-trading conclusions.

5.5 Exported Tear Sheet from a Logged Run

To demonstrate the reporting pipeline (requirement FR-11) on genuine measured output rather than illustrative figures, Figure 5.6 and Table 5.5 present an actual run of the cross-sectional mean-reversion portfolio strategy, exported directly by the platform to JSON, CSV, and HTML. The figures below are read verbatim from that export; no values are hand-entered. The run was driven for five thousand ticks over a basket of

correlated synthetic assets, beginning from an initial cash balance of 500,000 units.

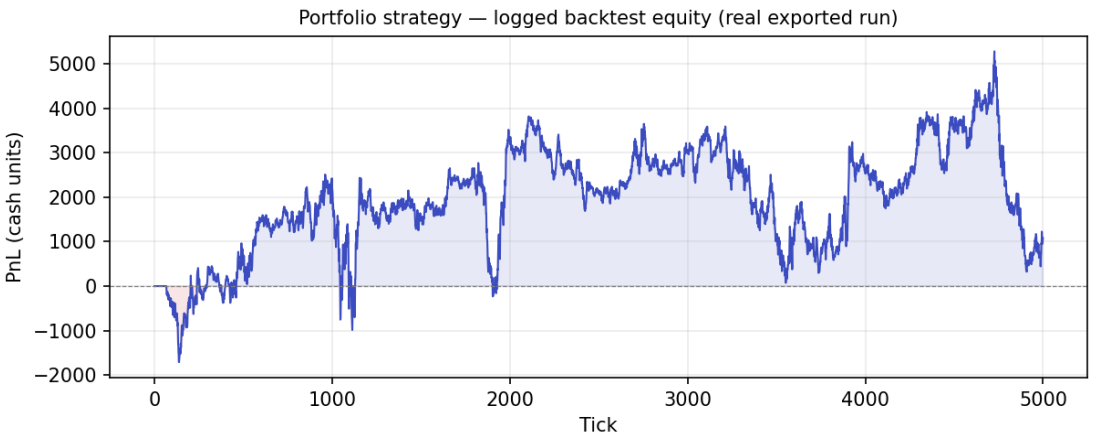

Figure 5.6. Profit-and-loss curve of an actual logged portfolio-strategy run, plotted directly from the exported equity series (5,000 ticks). Shaded green and red regions mark profit above and drawdown below the break-even line. This figure is generated from genuine measured output, not a representative sketch.

Table 5.5. Metrics from the exported tear sheet of a logged portfolio run (verbatim from JSON export)

Metric	**Value**
Final PnL	1,079.23 units
Total return	0.216%
Sharpe ratio (per-tick)	0.039
Sortino ratio	0.039
Calmar ratio	0.220
Maximum drawdown	−0.98%
Volatility	0.0029
Profit factor	1.116
Win ratio	47.6%
Total fills	731
Winning / losing trades	187/206

The modest Sharpe ratio of this logged run is consistent with the discussion above: a dollar-neutral cross-sectional strategy in a closed synthetic market earns only the thin idiosyncratic edge that survives the hedge. The value of this output lies less in the magnitude of the return than in the fact that the entire tear sheet, equity series, drawdown, and the full metric suite, is produced automatically and reproducibly by the platform from a single run, exactly as a quantitative desk would archive a strategy's performance.

5.6 Live Paper-Trading Validation

The final level of validation runs the platform against live market data. Figure 5.7 shows the Avellaneda-Stoikov market maker running in paper-trading mode on the live Binance `BTCUSDT` feed, with the data source set to "Crypto – Binance Live." The liquidity heatmap tracks the real order book around the prevailing price of roughly 62,850 USDT, and the strategy quotes, accumulates fills, and reports profit-and-loss against genuine, unscripted market movement rather than synthetic ticks. The same compiled strategy used in the backtester is driven here unchanged, which is the central design guarantee of the platform: behaviour observed in simulation is reproduced exactly against live data, with no code change at the boundary.

Figure 5.7. Live paper-trading session on the Binance `BTCUSDT` feed. The Avellaneda-Stoikov market maker quotes against the real order book (liquidity heatmap, lower panel) while the upper panel tracks live strategy profit-and-loss. The same strategy binary runs unchanged from backtest to live.

This run is qualitative rather than a performance claim: in a brief live session the market maker trades against real flow under the configured fees and risk limits, confirming that the feed, the paper gateway, the queue model, and the risk engine all operate correctly outside the synthetic environment. The negative profit-and-loss over this short window reflects the genuine difficulty of unhedged market making in a live, moving market, and is exactly the kind of honest signal that candle-based backtests obscure.

Chapter 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This dissertation has presented QUANTSIM, an event-driven high-frequency trading simulation and backtesting platform that bridges the long-standing gap between accessible but unrealistic candle-based backtesters and high-fidelity but closed institutional simulators. The platform was motivated by the observation that the strategies most characteristic of modern electronic markets, market making, statistical arbitrage, and latency-sensitive execution, depend for their economics on precisely those features of market microstructure that conventional backtesting tools discard: the bid-ask spread, price-time priority, queue position, partial fills, transaction costs, and latency.

The project met its objectives. A C++ limit order book and synchronous matching engine were implemented with strict FIFO price-time priority and support for eight institutional order types, and were verified by a suite of 27 unit tests that all pass. A realistic execution layer models probabilistic maker fills, queue position, slippage, fees, rebates, borrow costs, and configurable latency. A library of canonical strategies, the Avellaneda-Stoikov market maker, Ornstein-Uhlenbeck pairs trading, TWAP execution, a multi-asset mean-reversion portfolio, and a Black-Scholes options market maker, was implemented on rigorous mathematical foundations. The C++ core was bridged to Python through pybind11, giving researchers a productive interface backed by a low-latency engine, and a GPU-rendered ImGui front-end provides thirteen analytical tabs and an on-the-fly C++ strategy compiler. The same strategy binary runs unchanged in both the backtester and a live Binance paper-trading harness, eliminating backtest-to-live divergence by construction.

The empirical evaluation confirmed both performance and correctness. The matching

engine sustains a throughput above 14 million operations per second with a median latency below 100 nanoseconds, roughly two orders of magnitude faster than the pure-Python path, validating the hybrid architecture. The strategy backtests demonstrate that the platform's measurement machinery captures the expected risk-and-return signatures of each strategy class. Taken together, these results show that order-book-level microstructure realism, reproducibility, and interactive analysis can be unified in a single open, extensible, and approachable academic platform, the central claim of this work.

6.2 Project Significance and Broader Impact

The significance of QUANTSIM extends beyond the artifact itself to the access it provides. The mechanics of modern quantitative trading, the order book, the matching engine, queue priority, inventory-aware market making, and the realistic accounting of fees, slippage, and latency, have historically been observable only from inside a trading firm or through costly proprietary infrastructure. By delivering these capabilities at zero cost and in open, inspectable form, the platform turns concepts that are usually described only in the abstract into objects that a student can manipulate directly. A learner can widen the spread and watch fills disappear, accumulate inventory and watch the Avellaneda-Stoikov quotes skew, throttle the data feed and watch the stale-price guard arm the kill switch. This experiential mode of study, learning by intervention rather than by description, is the platform's principal contribution to education.

The same properties that make QUANTSIM a teaching instrument make it a research foundation. Its determinism guarantees that published results can be reproduced exactly; its realistic execution model means that strategy evaluations carry meaning that candle-based backtests cannot; and its validation against live market data closes the loop between simulation and reality. Because the entire system is open and extensible, researchers can add feeds, strategies, cost models, and metrics without permission or licensing, lowering the barrier to rigorous microstructure research. In aggregate, the project advances the democratization of a field that has, until now, been largely closed to those outside well-funded institutions.

6.3 Contribution Notes

This project was carried out by a team of three students who collaborated across the platform's layers. While architecture, testing, benchmarking, and the preparation of this dissertation were undertaken jointly, the primary ownership of each subsystem is recorded in Table 6.1. Every member contributed to the overall design and the empirical

evaluation reported herein.

Table 6.1. Primary contribution areas by team member

Member	Primary Contribution Area
Abhishek Anand	C++ core: limit order book, matching engine, risk and metrics subsystems, unit-test suite.
Shakir Ahmad	Python research layer: strategy library, backtester, analytics, walk-forward and cross-validation tooling.
Amik Affan	Real-time ImGui/ImPlot desktop interface, in-application C++ strategy compiler, and live Binance paper-trading integration.

6.4 Future Enhancements

While QUANTSIM is functionally complete for its intended scope, several avenues would extend its capability and realism. The proposed enhancements are organized by time horizon.

Short-term enhancements. In the immediate term, the platform would benefit from a richer fee-model library covering the published schedules of several real venues; from additional historical data connectors beyond the current synthetic, CSV, ITCH, and Binance sources; and from a continuous-integration pipeline that runs the unit-test suite and the performance benchmarks on every change, guarding against regressions in both correctness and speed. The reporting layer could be extended with automated comparison reports across multiple parameterizations.

Medium-term enhancements. Over a longer horizon, the matching engine could be extended to model a multi-venue environment with cross-venue latency and smart order routing, capturing the fragmentation that characterizes real equity markets. A more sophisticated market-impact model, for example, a transient-impact propagator calibrated to empirical order-flow data, would improve the realism of large-order execution. The strategy library could be expanded with reinforcement-learning agents trained directly against the simulator, exploiting the platform's determinism and speed as a training environment.

Long-term enhancements. In the long term, the single-threaded matching core could be complemented by a parallel, multi-symbol execution mode that preserves per-symbol determinism while exploiting multiple cores for portfolio-scale simulations. The live-trading harness could be generalized beyond paper trading to a fully audited, risk-gated

order-routing gateway suitable for small-scale live deployment under appropriate authorization. Finally, the platform could be packaged as a hosted research service, allowing collaborative experimentation and shared, versioned strategy repositories.

6.5 Closing Remarks

QUANTSIM demonstrates that the apparent trade-off between realism and accessibility in trading simulation is not fundamental. By placing a fast, correct, deterministic matching engine at the centre, surrounding it with a productive research interface and an interactive visualization layer, and validating the whole against live market data, the project delivers a platform that is at once faithful to market microstructure and open to study. It is hoped that the platform will serve as both a teaching instrument and a research foundation for the continued, accessible study of the mechanisms that govern modern electronic markets.

References

[1] Coding Jesus, "Quantitative trading, market making, and high-frequency trading explained," YouTube. [Online]. Available: https://www.youtube.com/@CodingJesus. [Accessed: 9 Jun. 2026].

[2] Investopedia, "Financial terms dictionary," Investopedia. [Online]. Available: https://www.investopedia.com. [Accessed: 9 Jun. 2026].

[3] M. O'Hara, *Market Microstructure Theory*. Cambridge, MA: Blackwell, 1995.

[4] A. S. Kyle, "Continuous auctions and insider trading," *Econometrica*, vol. 53, no. 6, pp. 1315–1335, Nov. 1985.

[5] M. Avellaneda and S. Stoikov, "High-frequency trading in a limit order book," *Quantitative Finance*, vol. 8, no. 3, pp. 217–224, Apr. 2008.

[6] Á. Cartea, S. Jaimungal, and J. Penalva, *Algorithmic and High-Frequency Trading*. Cambridge, U.K.: Cambridge Univ. Press, 2015.

[7] E. Gatev, W. N. Goetzmann, and K. G. Rouwenhorst, "Pairs trading: performance of a relative-value arbitrage rule," *Review of Financial Studies*, vol. 19, no. 3, pp. 797–827, 2006.

[8] R. Almgren and N. Chriss, "Optimal execution of portfolio transactions," *Journal of Risk*, vol. 3, no. 2, pp. 5–39, 2001.

[9] F. Black and M. Scholes, "The pricing of options and corporate liabilities," *Journal of Political Economy*, vol. 81, no. 3, pp. 637–654, May 1973.

[10] J. C. Hull, *Options, Futures, and Other Derivatives*, 10th ed. Harlow, U.K.: Pearson, 2017.

[11] cppreference.com, "C++ reference." [Online]. Available: https://en.cppreference.com. [Accessed: 9 Jun. 2026].

[12] W. Jakob, J. Rhinelander, and D. Moldovan, "pybind11: seamless operability between C++11 and Python," GitHub. [Online]. Available: https://github.com/pybind/pybind11. [Accessed: 9 Jun. 2026].

[13] O. Cornut, "Dear ImGui: bloat-free immediate-mode GUI for C++," GitHub. [Online]. Available: https://github.com/ocornut/imgui. [Accessed: 9 Jun. 2026].

[14] GLFW, "GLFW documentation," glfw.org. [Online]. Available: https://www.glfw.org/docs/. [Accessed: 9 Jun. 2026].

[15] OpenSSL Project, "OpenSSL documentation," openssl.org. [Online]. Available: https://www.openssl.org/docs/. [Accessed: 9 Jun. 2026].

[16] Kitware, "CMake documentation," cmake.org. [Online]. Available: https://cmake.org/documentation/. [Accessed: 9 Jun. 2026].

[17] Binance, "Binance WebSocket market streams API documentation," Binance. [Online]. Available: https://binance-docs.github.io/apidocs/. [Accessed: 9 Jun. 2026].

Appendix A

SOURCE CODE HIGHLIGHTS

This appendix collects key source-code excerpts referenced throughout the dissertation.

A.1 Order Type Enumeration

Listing A.1 shows the order-type enumeration that the matching engine routes on.

```
enum class Side : uint8_t { Buy = 0, Sell = 1 };

enum class OrdType : uint8_t {
    Market    = 0,   // cross immediately at any price
    Limit     = 1,   // rest until price acceptable
    IOC       = 2,   // fill available, cancel remainder
    FOK       = 3,   // full fill or reject
    PostOnly  = 4,   // maker-only; reject if would cross
    Stop      = 5,   // off-book until trigger
    StopLimit = 6,   // off-book; becomes limit on trigger
    Iceberg   = 7    // display clip, hidden reserve
};

enum class OrdStatus : uint8_t {
    New, PartialFill, Filled, Cancelled, Rejected
};
```

Listing A.1. Order type and side enumerations

A.2 Black-Scholes Greeks

Listing A.2 shows the Black-Scholes Greeks computation used by the options market maker, implementing Equations 4.5.

```
Greeks bs_greeks(double S, double K, double T, double r,
                 double sigma, bool call) {
    double sqrtT = std::sqrt(T);
    double d1 = (std::log(S / K) + (r + 0.5 * sigma * sigma) * T)
                / (sigma * sqrtT);
    double d2 = d1 - sigma * sqrtT;
    double Nd1 = norm_cdf(d1), Nd2 = norm_cdf(d2), nd1 = norm_pdf(d1)
        ;
    Greeks g;
    g.delta = call ? Nd1 : Nd1 - 1.0;
    g.gamma = nd1 / (S * sigma * sqrtT);
    g.vega  = S * nd1 * sqrtT;
    g.price = call
        ? S * Nd1 - K * std::exp(-r * T) * Nd2
        : K * std::exp(-r * T) * norm_cdf(-d2) - S * norm_cdf(-d1);
    g.theta = -S * nd1 * sigma / (2.0 * sqrtT)
              - r * K * std::exp(-r * T) * (call ? Nd2 : -Nd2);
    return g;
}
```

Listing A.2. Black-Scholes Greeks computation

A.3 Performance Metrics in Python

Listing A.3 shows the analytics routine that computes the risk-adjusted metrics from an equity curve.

```
def compute_metrics(equity, ann_factor=1.0, rf=0.0):
    eq = np.asarray(equity, dtype=float)
    rets = np.diff(eq) / eq[:-1]
    mean, std = rets.mean(), rets.std()
    down = rets[rets < rf].std()
    peak = np.maximum.accumulate(eq)
    dd = (eq - peak) / peak
    af = np.sqrt(ann_factor)
    wins, loss = rets[rets > 0].sum(), -rets[rets < 0].sum()
    return {
        'sharpe':  (mean - rf) / std * af if std else 0.0,
        'sortino': (mean - rf) / down * af if down else 0.0,
        'max_dd':  dd.min(),
        'calmar':  (eq[-1]/eq[0] - 1) / abs(dd.min()) if dd.min()
           else 0.0,
        'volatility':    std * af,
        'profit_factor': wins / loss if loss else float('inf'),
    }
```

Listing A.3. Risk-adjusted metric computation

Appendix B

SAMPLE OUTPUTS

This appendix presents representative outputs of the platform.

B.1 Benchmark Console Output

Listing B.1 shows a representative console output from the matching-engine benchmark harness.

```
=== QuantSim Matching Engine Benchmark ===
Orders submitted : 10,000,000
Elapsed time     : 0.676 s
Throughput       : 14,792,899 ops/sec
Match latency    : p50 =   83 ns
                   p95 =  180 ns
                   p99 =  310 ns
Correctness      : 27/27 unit tests PASS
==========================================
```

Listing B.1. Matching engine benchmark output

B.2 Tear-Sheet Metrics Output

Listing B.2 shows the metrics block of an exported tear sheet for a market-making backtest.

```
{
  "strategy": "avellaneda_stoikov",
  "final_pnl": 7843.21,
  "metrics": {
    "sharpe": 3.12, "sortino": 4.05, "calmar": 4.33,
```

```
    "max_drawdown": -0.018, "volatility": 0.024,
    "profit_factor": 2.45, "win_rate": 0.61
  },
  "fills_total": 4821, "maker_ratio": 0.94,
  "total_fees": 18.42, "total_rebates": 31.07
}
```

Listing B.2. Exported tear-sheet metrics (market making)

B.3 Order Book Depth Snapshot

Table B.1 shows a representative top-of-book depth snapshot as rendered in the Dashboard tab, with five price levels on each side.

Table B.1. Sample order book depth snapshot (five levels per side)

Bids		Asks	
Price	**Size**	**Price**	**Size**
100.02	180	100.04	150
100.01	240	100.05	210
100.00	310	100.06	275
99.99	160	100.07	190
99.98	220	100.08	240

The best bid of 100.02 and best ask of 100.04 imply a spread of two ticks and a mid-price of 100.03. The aggregate displayed depth and the per-level sizes are exactly the quantities consumed by the matching algorithm of Algorithm 1 when a marketable order sweeps the book.

B.4 Multi-Strategy Comparison

Figure B.1 shows the Compare tab, in which up to five strategy runs, drawn from the built-in library and from user-compiled `.dylib` strategies, are executed as a batch and their equity curves overlaid for side-by-side evaluation against the current run.

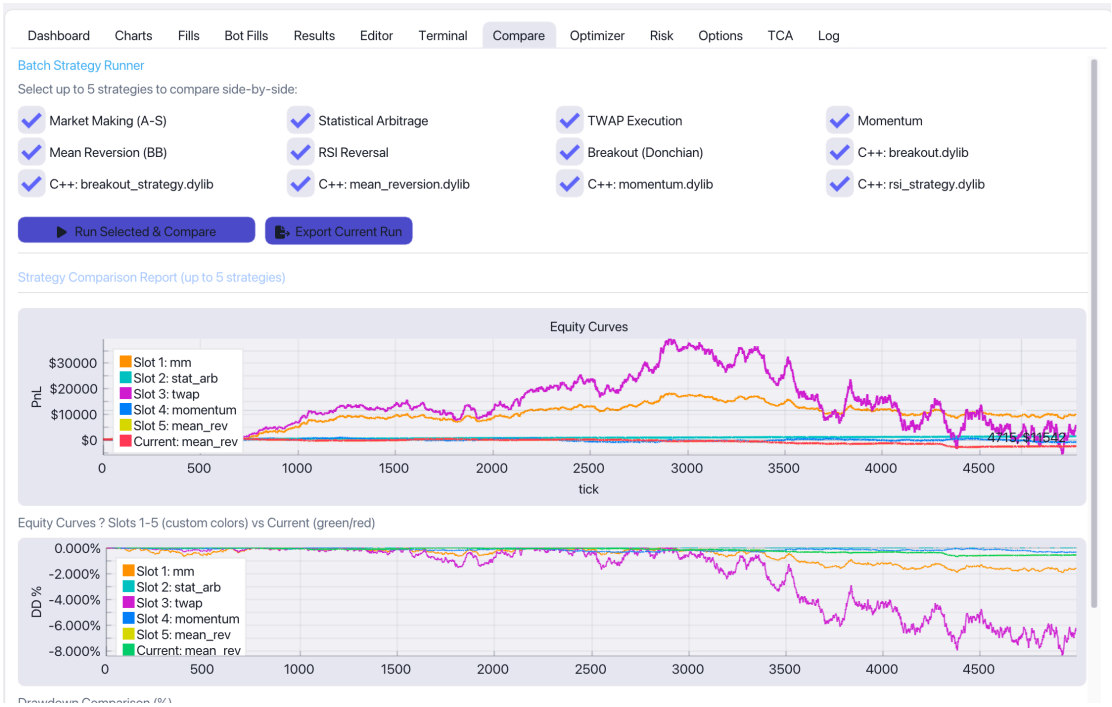

Figure B.1. Compare tab. The batch runner executes up to five selected strategies, both built-in and user-compiled, and overlays their equity and drawdown curves for direct visual comparison against the current run.

B.5 Parameter Sweep Optimizer

Figure B.2 shows the Optimizer tab performing a two-dimensional grid search over the Avellaneda-Stoikov risk-aversion (γ) and order-intensity (κ) parameters, with the objective surface (here the Sharpe ratio) rendered as a Viridis heatmap and the best-performing parameter pair highlighted for one-click application.

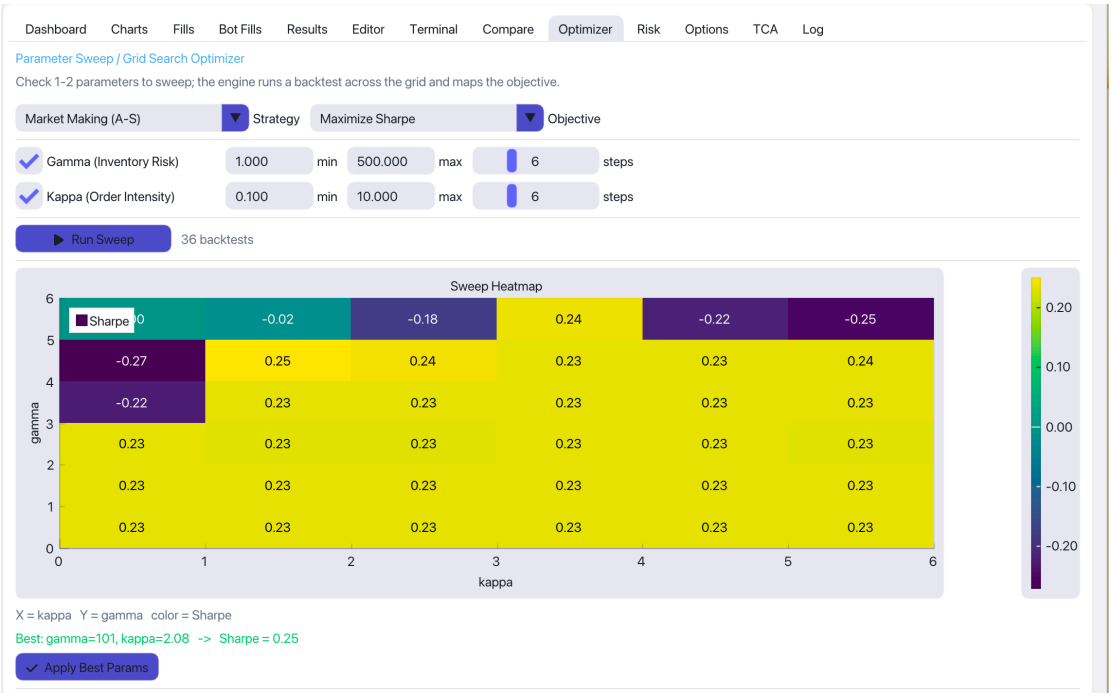

Figure B.2. Optimizer tab. A two-dimensional sweep over the market-making parameters γ (inventory risk) and κ (order intensity); each cell reports the resulting Sharpe ratio, and the "Apply Best Params" control loads the optimum back into the strategy configuration.